A Study on Robust Quantitative Techniques for Capital Allocation

Capstone Project

Vinayak B. Menon

August 28, 2020

Abstract

The mean-variance optimization framework introduced by Markowitz, has been a significant contribution to the field of quantitative finance. Yet the presence of high estimation errors can lead to diverging results. In practice, this would lead to a high turnover rate when managing portfolios. This instability could be present to a point that it offsets the benefits of diversification.

In this report, our aim is to discuss a few methods that attempt to resolve issues related to instability. We discuss both commonly known methods as well as recently developed methods. We finally compare these techniques, using both simulated and actual data, based on their overall performance.

Table of Contents

List of Tables **iv**

List of Symbols and Abbreviations **v**

1 Introduction **1**

2 Problem Statement **3**

3 Theoretical Background **5**

3.1 Instability due to Noise 6

3.1.1 Marcenko-Pastur Distribution 7

3.2 Instability due to Signal 8

4 Methods to Control for Instability **10**

4.1 Critical Line Algorithm (CLA) 11

4.1.1 Stage 1: Obtaining the Starting point 13

4.1.2 Stage 2: Iteration Process 14

4.2 Treatment on the Covariance Matrix 17

4.2.1 The Linear Shrinkage Method 17

4.2.2 Eigen Value Clipping Method 18

4.2.3 Targeted Shrinkage Method 20

4.3 Hierarchical Risk Parity (HRP) 20

4.3.1 Stage 1: Tree Clustering 22

4.3.2 Stage 2: Quasi Diagonalization 24

4.3.3 Stage 3: Recursive Bisection 25

4.4 Nested Clustered Optimization Method (NCO) 26

5 Monte Carlo Optimization Selection (MCOS) **28**

6 Numerical Experiments **30**

6.1 Test 1: Comparison of Denoising techniques 31

6.2 Test 2: Comparison of Optimization Techniques on Simulated Data . . . 33
6.3 Test 3: Comparison of Optimization techniques on Actual Data 34

7 Conclusion **35**

References

A Appendix

List of Tables

1 Denoising techniques comparison based on standard deviation of RMSE ($\times 10^{-3}$)- Minimum Variance Portfolio 32

2 Denoising techniques comparison based on standard deviation of RMSE ($\times 10^{-3}$)- Maximum Sharpe Ratio Portfolio 33

3 Comparison of Optimization techniques on Simulated Data based on standard deviation of RMSE and Return difference ($\times 10^{-3}$) 34

4 Comparison of Optimization techniques on Actual Data 35

List of Symbols and Abbreviations

MPT: Modern Portfolio Theory

MVO: Mean Variance Optimization

CLA: Critical Line Algorithm

HRP: Hierarchial Risk Parity method

NCO: Nested Cluster Optimization method

MCOS: Monte Carlo Optimization Selection method

X: Return data of N assets with T number of observations

μ: Expected return of X

$\bar{\mu}$: Estimated expected return of X

Σ: Covariance matrix of X

$\bar{\Sigma}$: Estimated covariance matrix of X

ω^*: Optimal weight vector

$\bar{\omega}^*$: Estimated ptimal weight vector

a: Characteristic vector defining the portfolio constraints

l: Lower bound constraint on ω

u: Upper bound constraint on ω

1 Introduction

Portfolio optimization is an integral component of investment management and a recurring topic of interest in financial research. Harry Markowitz formalized this concept effectively through a mathematical framework, now known as Modern Portfolio Theory(MPT) or Mean-Variance Analysis. This theoretical framework provided great insights on how to obtain optimal portfolios for various levels of risk, giving rise to the notion of an 'efficient frontier' for a collection of assets [3]. As per MPT, the primary goal of portfolio optimization is to construct a portfolio that provides the highest possible return for a given level of risk, or, to produce a portfolio of minimum risk given a level of expected return. Mathematically, this problem can be incorporated as a quadratic convex optimization problem, and is commonly referred to as a mean-variance optimization problem(MVO), or the Markowitz optimization framework.

In terms of algorithmic complexity, convex optimization problems have the benefit of being a P problem, or a polynomial time problem. This implies that there exists an algorithm that can solve such a problem within computational limits. Because of this, standard optimization procedures exist that can solve these types of quadratic optimization problems. The Critical Line Algorithm (CLA) is an algorithm outside of this collection, introduced by Markowitz himself, which was specifically suited for application in Mean-Variance Optimization that includes linear inequalities. [3, 4].

Yet even with the existence of such methods for optimal capital allocation, there are a number of issues that arise when applied in practice. One major drawback lies in their high sensitivity to input errors i.e a small change in the input would result in an extreme change in the output. When managing portfolios, this can be a huge issue, as it can lead to a high turnover rate in capital allocation , which will have associated turnover costs. Given the nature of financial data, we constantly deal with estimates and not true values and naturally, a small margin of error is to be expected. It would be of no surprise then that portfolios that are optimized in-sample fail to outperform or even match the naive weighted allocation of a portfolio out of sample [7, 8, 10]. Thus, the need for robust optimization techniques is evident.

The Critical Line Algorithm (CLA) faces this issue of instability as well and a considerable amount of research has been done in order to find a resolution. In order to circumvent this issue, some practicioners have attempted to rely on methods that solely deal with the covariance structure rather than the mean to remove this drawback, giving rise to what are commonly known as risk-parity portfolios [9]. Yet such risk based methods only reduce these instability issues rather than resolve them.

There are two main inputs when performing portfolio optimization: the return expectations of the assets under consideration and the covariance matrix of the return series, containing the variance of returns for an an asset and any correlations between different asset returns. Our focus in this report will be on instabilities caused by the covariance structure for a given collection of assets. The instabilities due to the covariance matrix can be attributed to two causes [2]. First, the presence of noise in the input parameters can lead to inaccurate solutions. Pre-processing the covariance matrix before attempting optimization usually solves this issue. Second, the signal structure may exist in such a way that it magnifies the estimation errors significantly, again leading to widely deviating results. This is an relatively difficult issue to solve and requires treatment in the optimization process itself.

This report attempts to discuss conventional and recently developed techniques in portfolio optimization that targets the instabilities arising due to the covariance matrix. Our goal is to identify those techniques that provide stable results and how they avoid instability. We will first describe the Markowitz optimization problem framework along with the associated variables and constraints in Section 2. We then move on to Section 3 for a technical and theoretical background required moving ahead. After this, we discuss few methods that attempt to resolve instability issues. The methods discussed are not an exhaustive list and are a mixture of both commonly known and recently developed techniques. After discussing these techniques, we briefly describe the Monte Carlo Optimization Selection method (MCOS) [1] in Section 5, and how it can be used to judge the robustness of a given method.

We compare each of the previously discussed methods using the MCOS method dis-

cussed in Section 6 with generated and actual data. The data and methodology used will be discussed here, as well as an analysis of the results, after which we conclude our report in Section 7 with a summary of our findings.

2 Problem Statement

Consider the return data, X, of N assets with T number of observations. We will define the array of expected return of the N assets to be μ and the covariance matrix of the returns to be Σ. We consider the classical asset allocation problem framed by Markowitz. The goal of our optimization problem would be to obtain the optimal weight vector ω^* that minimizes the variance of the system, mathematically denoted as $\omega^{'}\Sigma\omega$, while adjusting for any portfolio constraints. The problem can be framed as:

$$\begin{aligned} &\min_{\omega} \frac{1}{2}\omega'\Sigma\omega \\ &\text{s.t } \omega'a = 1 \end{aligned} \tag{1}$$

where a characterizes the portfolio constraints. Upon applying the Lagrangian, we modify the above problem into an unconstrained form as follows:

$$\min_{\omega} L\left[\omega, \lambda\right] = \frac{1}{2}\omega'\Sigma\omega - \lambda\left(\omega'a - 1\right) \tag{2}$$

Using the Karush-Kuhn-Tucker first order necessary conditions [11] for an optimal solution on (2), we have:

$$\frac{\partial L}{\partial \omega} = \Sigma\omega - \lambda a = 0 \Rightarrow \omega = \lambda\Sigma^{-1}a \tag{3}$$

$$\frac{\partial L}{\partial \lambda} = \omega'a - 1 = 0 \Rightarrow \omega'a = a'\omega = 1 \tag{4}$$

Combining (3) and (4), we get:

$$\Rightarrow \frac{\partial L}{\partial \lambda} = \lambda a'\Sigma^{-1}a = 1 \Rightarrow \lambda = \frac{1}{a'\Sigma^{-1}a} \tag{5}$$

Substituting (5) in (3), we obtain:

$$\omega^* = \frac{\Sigma^{-1}a}{a'\Sigma^{-1}a} \tag{6}$$

Now applying the Karush-Kuhn-Tucker second order sufficient conditions [11] for a minimum on (2), we have:

$$\begin{vmatrix} \dfrac{\partial L^2}{\partial \omega^2} & \dfrac{\partial L^2}{\partial \omega \partial \lambda} \\ \dfrac{\partial L^2}{\partial \lambda \partial \omega} & \dfrac{\partial L^2}{\partial \lambda^2} \end{vmatrix} = \begin{vmatrix} \Sigma' & -a' \\ a & 0 \end{vmatrix} = a'a \geq 0 \tag{7}$$

The optimal allocation vector of weights can thus be written in closed form as $\omega^* = \dfrac{\Sigma^{-1}a}{a'\Sigma^{-1}a}$.

The convex optimization solution, obtained in (6), is the simplest direct approach in solving the standard portfolio optimization problem with only linear equalities. However, the lack of any bounds on the weights means you have little control over the solutions that leading to extreme holdings and no restriction on shorting. To resolve this, it is generally useful to include linear inequality constraints as follows:

$$\begin{aligned} &\min_{\omega} \frac{1}{2}\omega'\Sigma\omega \\ &\text{s.t } \omega'a = 1 \\ &l \leq \omega \leq u \end{aligned} \tag{8}$$

where $l = [l_1, l_2, \ldots l_N]$ and $u = [u_1, u_2, \ldots u_N]$ are the lower and upper bounds, respectively, on the weight vector ω.

Unfortunately, the inclusion of the inequalities makes it difficult to solve the original problem as the standard Lagrangian method cannot be used in the presence of inequality constraints to obtain a closed form solution. Numerical methods will be required to obtain approximate solutions, although the CLA has been known to provide a way around this dilemma.

It will be useful to note that, depending on the the form of the characteristic vector a, we can obtain different portfolios, namely:

1. When $a = 1_N$, the solution is the minimum variance portfolio.

2. When $a = \mu$, we obtain the portfolio with maximum Sharpe Ratio.

Our problem of focus in this report will be (8) i.e we include constraints on the weights with $l = 0_N$ and $u = 1_N$, thus preventing shorting.

Now that we have defined the optimization framework to be investigated and discussed its basic properties, we can move on to discussing the causes for numerical instability in such problems as well as the theory required to understand them.

3 Theoretical Background

Typically, the true values for a and Σ are unknown and we rely on estimates of these values which we denote here as $\bar{a}$ and $\bar{\Sigma}$ respectively.

It is here where we encounter the main problem that we are attempt to investigate in this report, namely the numerical instability of of such solutions. These approaches tend to be very sensitive to the inputs provided. Since it is impossible to obtain true values, we rely on estimates, which would lead to significantly different results compared to the true result. In practice, this is a major issue when trying to obtain $\bar{\omega}^*$, since there are material costs associated with constant reallocation of assets with different solutions.

The two inputs that are necessary for portfolio optimization is the mean vector, $\bar{\mu}$, and the covariance matrix, $\bar{\Sigma}$. Although instability can be attributed to sensitivities towards both of these inputs, our focus will be towards the covariance structure in particular. This is because the covariance matrix ultimately holds the key information required for capital allocation i.e correlations, which is also the primary reason for introducing portfolio optimization in the first place.

Instability of solutions in convex optimization problems can be associated with two causes [2], namely:

1. Instability due to noise

2. Instability due to structure

We will proceed to discuss each of these causes in depth.

3.1 Instability due to Noise

Covariance matrices are an essential part of portfolio construction. This is because, given a collection of N random variables with T rows of observations, the covariance matrix obtained contains information regarding the co-movement between these random variables. This naturally becomes helpful in creating a portfolio that can generate returns while softening any negative impacts.

As mentioned before, the inverse of the covariance matrix, $\bar{\Sigma}^{-1}$ is ultimately required when performing portfolio optimization. Assuming we are working on standardized variables (the covariance matrix will simply be the correlation matrix), the formula for the inverse is as follows:

$$\bar{\Sigma}^{-1} = \frac{1}{|\bar{\Sigma}|} \begin{bmatrix} 1 & -\rho \\ -\rho & 1 \end{bmatrix}$$

As we can see, the term $|\bar{\Sigma}|$ has a huge effect on our results, as $|\bar{\Sigma}| \to 0$ implies that $\bar{\Sigma}^{-1}$ will explode.

The condition number of a matrix A describes the magnification errors in a given system of equations. In other words, it measures the instability or the variation of output given a small change in the input. Using the eigen value decomposition of the covariance matrix $\bar{\Sigma}$, we can obtain its eigen values $\{\lambda_i\}_{i=1,2,...N}$. The condition number can then be denoted mathematically as:

$$\kappa(\bar{\Sigma}) = \frac{|\lambda_1|}{|\lambda_N|}$$

i.e the ratio between the absolute values of the maximal and minimal eigen values. The higher the condition number is, the more ill conditioned the matrix is said to be. In other words, a matrix with a high condition number will more likely lead to magnification of errors. Many methods have been introduced to manipulate this condition number.

Shrinkage techniques are a prominent class of techniques that has been used consistently to treat the covariance matrix, where the covariance matrix is shrunk to reduce the condition number [12].

Unfortunately, given the nature of the provided data as well as how it is handled and presented, a considerable amount of noise is likely to exist along with the signal required. The presence of such noise can then lead to huge errors due to the reason above. Most shrinkage techniques, although beneficial, does not have any process that removes noise from signal and thus may effectively be shrinking the signal as well. In fact, the noise may affect the whole optimization process to the point that the results are very well useless in practice. Considering the finite and non deterministic nature of financial data, it is highly likely that we are working with ill conditioned covariance matrices, if not treated properly. Thus a logical approach to resolving this issue can be to remove any noise present in the inputs so as to prevent any magnification of their influence. We discuss an approach below that can help in this goal.

3.1.1 Marcenko-Pastur Distribution

Consider a matrix X of size $T \times N$, consisting of independent and identically distributed random observations from a distribution $D(0, \sigma)$, with mean 0 and variance σ. The covariance matrix can then be obtained as:

$$C = T^{-1} X^{'} X$$

According to the Marcenko-Pastur Theorem [13], as $N \to +\infty$ and $T \to +\infty$ with $1 < \frac{T}{N} < +\infty$, the eigen values λ of C asymptotically converge to the following distribution, commonly known as the Marcenko-Pastur distribution:

$$f[\lambda] = \begin{cases} \frac{T}{N} \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{2\pi\lambda\sigma^2} & \text{if } \lambda \in [\lambda_-, \lambda_+] \\ 0 & \text{if } \lambda \notin [\lambda_-, \lambda_+] \end{cases} \tag{9}$$

where the maximum expected eigen value is $\lambda_+ = \sigma^2 \left(1 + \sqrt{N/T}\right)^2$ while the minimum expected eigen value is $\lambda_- = \sigma^2 \left(1 - \sqrt{N/T}\right)^2$.

This result holds true for random matrices in general. Since the eigen values of a random generated matrix, i.e a matrix of independent and identically distributed collection of observations, would follow the above distribution, those that do not fit the distribution would be part of some signal rather than random behaviour. In other words, eigen values that belong in the interval $[\lambda_-, \lambda_+]$ would be attributed to noise whereas those that do not belong to the interval would be attributed as part of a signal to act on.

This brings a new dimension regarding how one can treat the covariance matrix. By being able to dissect the covariance structure such that we can segregate noise from signal, we can easily apply techniques to treat the noise explicitly without interfering with useful information. In this way, the Marcenko-Pastur Theorem allows us to clean our covariance matrix, allowing for less estimation errors as a result.

3.2 Instability due to Signal

Now that we have discussed the influence of noise with regards to the instability and a rule that can help us distinguish it from information, we move on to the next cause for instability i.e instability arising from the signal itself. Unlike the previous cause for covariance instability, signal induced instability cannot be solved by adjusting the $\frac{N}{T}$ ratio. In other words, increasing sample size will not regulate signal induced instability. This is because signal induced instability is related to the structure of the signal itself rather than the signal to noise ratio. To understand this further, we introduce the notion of condition number of a matrix, specifically the covariance matrix.

Let us consider a correlation matrix C between the returns of two assets, of the form:

$$C = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

where ρ is the correlation between the returns of the two assets. The correlation matrix

can then be diagonalized as follows. The eigen value equation for C can be written as:

$$|C - I\lambda| = 0$$

$$\Rightarrow C = \begin{vmatrix} 1-\lambda & \rho \\ \rho & 1-\lambda \end{vmatrix} \Rightarrow \left(1-\lambda^2\right) - \rho^2 = 0$$

$$\Rightarrow (1-\lambda) = \pm\rho \Rightarrow \lambda = 1 \pm \rho$$

Upon finding the eigen values for C, we can form the diagonal matrix consisting of the eigen values thus obtaining the first part of the eigen decomposition. Without loss of generality, the diagonal matrix would then be of the form:

$$\Lambda = \begin{bmatrix} 1+\rho & 0 \\ 0 & 1-\rho \end{bmatrix}$$

To find the orthogonal matrix of eigen vectors, W, we solve the following equation:

$$\begin{bmatrix} 1-\lambda & \rho \\ \rho & 1-\lambda \end{bmatrix} \begin{bmatrix} W_{1,1} & W_{1,2} \\ W_{2,1} & W_{2,2} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Assuming C is not a diagonal matrix (i.e $\rho \neq 0$), we have:

$$W = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}$$

Now that we have obtained both Λ and W, we have completed the diagonalization process of the correlation matrix C, obtaining the result:

$$C = W\Lambda W'$$

We mention a few key points below:

1. $Tr(C) = \lambda_1 + \lambda_2 = 2$

2. $|C| = \lambda_1 \lambda_2 = 1 - \rho^2$

3. $C^{-1} = \frac{1}{|C|} \begin{bmatrix} 1 & -\rho \\ -\rho & 1 \end{bmatrix}$

We can see that the determinant of C i.e $|C|$ reached its maximum in the uncorrelated scenario where $\rho = 0$ (or when $\lambda_1 = \lambda_2 = 1$) whereas it reaches its minimum in the perfectly correlated case i.e $\rho = 1$ (or when either $\lambda_1 = 0$ or $\lambda_2 = 0$). Since the trace of C is a constant, we can see that as $|\rho| \to 1$, the ratio between λ_1 and λ_2 increases, with $|C|$ moving towards 0. Combining this inference with (3.), we can further state that as the two assets approach the state of perfect dependence, C^{-1} explodes. In other words, the condition number increases as the assets become more dependent.

In general, when we consider a portfolio of highly correlated assets, the condition number of the associated correlation matrix becomes extremely high, leading to an extremely ill-conditioned covariance matrix which further leads to extremely unstable optimal allocations. The Markowitz solution for portfolio allocation is then stable only if $\rho \approx 0$, which is essentially useless as the Markowitz solution is useful only when we deal with moderate to highly correlated assets. This dilemma is commonly known as the Markowitz's Curse. Because this issue lies at the heart of our goal i.e to be able to find optimal allocations given any collection of assets with varying correlations, it is a difficult issue to resolve. But with the advancement of machine learning techniques and the use of modern mathematics, methods have arisen that has attempted to resolve this problem in one way or another.

4 Methods to Control for Instability

Having discussed the necessary background behind the root causes for instability in mean variance optimization problems, we now look into methods that have been created to tackle these issues. Before we proceed with methods that attempt to resolve instability issues, it will be useful to understand the Critical Line Algorithm and its effectiveness in

portfolio optimization.

4.1 Critical Line Algorithm (CLA)

This inclusion of the inequalities makes it difficult to solve the original unconstrained problem as the Lagrangian method cannot be used effectively in the presence of inequality constraints. To resolve this issue, Markowitz introduced the CLA algorithm, a technique that effectively manages to circumvent the KKT conditions required for an optimum solution [3]. The significant contribution by the CLA algorithm is that it guarantees the exact solution after a known number of iterations [19]. Considering these points, we will describe the necessary background and algorithm in this section.

To understand the CLA algorithm, it is important to discuss the notion of free assets. Let $\mathbb{N}$ be the set of of indices in $1, 2, \ldots . N$ representing an asset in the considered universe. We define a set $\mathbb{F}$ to be the subset of $\mathbb{N}$ containing the asset indices of those assets whose weights are within the prescribed bounds. In other words, an asset whose index is k is said to be a free asset if

$$k \in \mathbb{F} = \{i \in \mathbb{N} : l_i \leq \omega_i \leq u_i\}$$

We also define $\mathbb{B} = \{i \in \mathbb{N} : \omega_i = l_i \text{ or } \omega_i = u_i\}$. This naturally implies that $\mathbb{N} = \mathbb{B} \cup \mathbb{F}$. Essentially, we have divided assets based on whether their weights are within or at their bounds (under the assumption that these weights have been obtained). We can further subdivide the set $\mathbb{B}$ as

$$\mathbb{B} = \mathbb{U} \cup \mathbb{L}$$

where $\mathbb{U} = \{i \in \mathbb{N} : \omega_i = u_i\}$ and $\mathbb{L} = \{i \in \mathbb{N} : \omega_i = l_i\}$. Let $k = |\mathbb{F}|$ Given this partition of $\mathbb{N}$, it is possible to partition the mean vector μ, the covariance matrix Σ and the weight allocation ω as follows:

$$\Sigma = \begin{bmatrix} \Sigma_F & \Sigma_{FB} \\ \Sigma_{BF} & \Sigma_B \end{bmatrix} ; \quad \mu = \begin{bmatrix} \mu_F \\ \mu_B \end{bmatrix} ; \quad \omega = \begin{bmatrix} \omega_F \\ \omega_B \end{bmatrix}$$

where Σ partitions as the $k \times k$ matrix Σ_F, $k \times (n-k)$ matrix Σ_{FB}, $(n-k) \times k$ matrix Σ_{BF} and the $(n-k) \times (n-k)$ matrix Σ_B. The mean vector μ is partitioned to μ_F of order $k \times 1$ and μ_B of order $(n-k) \times 1$, while the weight vector ω is partitioned as ω_F and ω_B of order $k \times 1$ and $(n-k) \times 1$ respectively. Note that $\Sigma_{FB} = \Sigma'_{BF}$ since Σ is symmetric by nature.

We shall refer to the solution to problem defined in (8) as the constrained minimum variance portfolio and the graphical representation of the set of solutions in the mean-variance plane as the constrained minimum variance frontier (CMVF).

A constrained minimum variance portfolio is said to be a turning point if, in its vicinity, other constrained minimum variance portfolios have different free assets. This is important to note as the entire CLA algorithm functions on the use of turning points.

When we know the set of free assets for a given level of expected return μ_p, i.e when we know $\mathbb{F}$ to the corresponding expected return, we can easily formulate the problem through the Lagrangian approach. We denote this in the proposition below.

Proposition 1: *The free weights in the solution ω^* of the constrained case are equal to the weights in the solution of the unconstrained case with the Lagrange function*

$$\begin{aligned} L = &\frac{1}{2}\omega'_F\Sigma_F\omega_F + \frac{1}{2}\omega'_F\Sigma_{FB}\omega_B + \frac{1}{2}\omega'_B\Sigma_{BF}\omega_F + \frac{1}{2}\omega'_B\Sigma_B\omega_B \\ &-\gamma\left(\omega'_F 1_F + \omega'_B 1_B - 1\right) - \lambda\left(\omega'_F\mu_F + \omega'_B\mu_B - \mu_P\right) \end{aligned} \tag{10}$$

where the components ω_F is subject to minimization while the components ω_B are fixed to their actual values.

We also add the following corollary.

Corollary 1: *Combining two neighboring turning points with a real weight $\alpha \in [0,1]$ always leads to a constrained minimum variance portfolio.*

Differentiating 10 with respect to ω_F, we have:

$$\Sigma_F\omega_F + \Sigma_{FB}\omega_B - \lambda\mu_F = \gamma 1_F$$

Incorporating the constraint $\omega' 1 = 1$ in terms of its partition, we will get

$$\gamma = -\lambda \frac{1_F' \Sigma_F^{-1} \mu_F}{1_F' \Sigma_F^{-1} 1_F} + \frac{1 - 1_B' \omega_B + 1_F' \Sigma_F^{-1} \Sigma_{FB} \omega_B}{1_F' \Sigma_F^{-1} 1_F}$$

Thus λ decides the value of γ and, by extension, the expected return of the minimum variance portfolio. In other words, the optimal solution is determined by the choice of λ. It is useful to note that λ and $\mu'\omega$ (or μ_P) are linearly related between turning points, implying that a higher value of λ will generate a constained minimum variance portfolio with higher expected return. We state this as a proposition below.

Proposition 2:*Between two turning points, λ and $\mu' w$ are linearly related, with a positive slope given as:*

$$\frac{\partial \left(\mu' \omega \left(\lambda\right)\right)}{\partial \lambda} \tag{11}$$

The proof of Proposition 1, Proposition 2 and Corollary 1 can be obtained from [20].

Now that we have discussed the necessary background, we move on to discussing the stages of the algorithm. The algorithm can be divided into two stages. We discuss each stage below.

4.1.1 Stage 1: Obtaining the Starting point

The first part of the algorithm involves finding the initial solution of the CMVF, which will be the turning point with the highest expected return. The steps for this stage can be summarized as follows:

1. Order all assets in descending order of their expected returns. In this representation, $n = 1$ will represent the asset with highest expected return while $n = N$ will represent the asset with lowest return.

2. Set all asset weights to their lower bounds i.e $\omega = l$ or $\omega_i = l_i, \forall i = 1, 2, \ldots N$. Let $i = 1$.

3. Stop if $\omega' 1 = 1$. Else

   - Increase the asset weight of the asset, ω_i.
   - If $\omega_i = u_i$, assign i=i+1.

The essence of this stage is that we are trying to find the smallest collection of assets (consisting of highest returns) which satisfies the full investment constraint. The resulting vector of weights is then a turning point, with the last modified weight being the free asset. Naturally the remaining weights lie in $\mathbb{B}$ since they either match the upper bounds or are still the lower bounds. The weight of the free asset, whose weight is denoted as ω_f, will be

$$\omega_f = 1 - \sum_{i \in \mathbb{U} \cup \mathbb{L}} \omega_i$$

.

4.1.2 Stage 2: Iteration Process

Now that we have obtained the starting turning point, the next stage involves the steps in the iterative process. The goal is to move from the initial turning point to the next turning point by decreasing the parameter λ. This can be done in 2 ways: either an asset in $\mathbb{F}$ moves to $\mathbb{B}$ or an asset in $\mathbb{B}$ moves to $\mathbb{F}$, either of which will confirm that a new turning point has been obtained as per the definition. We will discuss both cases here.

Case 1: Free asset to a Bounded asset

Consider the set of free assets $\mathbb{F}$ slightly below the turning point associated with λ_c i.e for λ where $\lambda_{t+1} < \lambda < \lambda_t = \lambda_c$. For this subset (containing k variables), we have:

$$\omega_F = -\Sigma_F^{-1}\Sigma_{FB}\omega_B + \gamma\Sigma_F^{-1}1_F + \lambda\Sigma_F^{-1}\mu_F$$

Substituting for γ, we can obtain ω_F as a linear function of λ. When $\frac{d\omega_i}{d\lambda} > 0$, decreasing λ will ω_i to l_i (i.e the asset i will move to the lower bound), whereas $\frac{d\omega_i}{d\lambda} < 0$ will force ω_i

to move to u_i as we decrease λ. We shall denote λ^i as the value of *lambda* for which ω_i moves to the corresponding bound.

We can derive λ^i from the previous steps to give:

$$\lambda^i = \frac{1}{C_i}\left[\left(1 - 1'_B\omega_B + 1'_F\Sigma_F^{-1}\Sigma_{FB}\omega_B\right)\left(\Sigma_F^{-1}1_F\right)_i - \left(1'_F\Sigma_F^{-1}1_F\right)\left(b_i + (\Sigma_F^{-1}\Sigma_{FB}\omega_B)_i\right)\right]$$

with:

$$C_i = \left(1'_F\Sigma_F^{-1}1_F\right)\left(\Sigma_F^{-1}\mu_F\right)_i + \left(1'_F\Sigma_F^{-1}mu_F\right)\left(\Sigma_F^{-1}1_F\right)_i$$

and

$$b_i = \begin{cases} u_i & if C_i > 0 \\ \\ l_i & if C_i < 0 \end{cases}$$

Note that for $k = 1$, we have $C_i = 0$, reflecting:

$$1'_F\omega_F = 1 - 1'_B\omega_B = w_i$$

This shows that the current case should only be considered when $k > 1$.

The next $\lambda < \lambda_c$, where an asset wants to leave the set $\mathbb{F}$, is:

$$\lambda_{inside} = \max_{i\in\mathbb{F}}\{\lambda^i\}$$

or λ_{inside} does not exist $\forall i$, owing to the above mentioned situations where $C_i = 0$. However, λ_{inside} describes the next turning point only if there is no portfolio with a λ, where $\lambda_{inside} < \lambda < \lambda_c$ and where an asset at its bounds wants to enter the subset $\mathbb{F}$. This situation is summarized in the next case.

Case 2: Bounded asset to a Free asset

As we mentioned above, a situation may occur where, as we move down in $\mu'\omega$, a formerly bound asset may wish to be a free asset. The corresponding portfolio therefore

becomes a turning point with subsets $\mathbb{F}$ and $\mathbb{B}$ redefined. We denote the new subsets as:

$$\mathbb{F}_i \equiv \mathbb{F} \cup i$$

$$\mathbb{F}_i \equiv \mathbb{F}/i$$

where $i \in \mathbb{B}$. The value λ^i where the weight of the newly included asset i moves away from its bound is given by:

$$\lambda^i = \frac{1}{C_i}\left[\left(1 - 1'_{B_i}\omega_{B_i} + 1'_{F_i}\Sigma_{F_i}^{-1}\Sigma_{F_iB_i}\omega_{B_i}\right)\left(\Sigma_{F_i}^{-1}1_{F_i}\right)_i - \left(1'_{F_i}\Sigma_{F_i}^{-1}1_{F_i}\right)\left(b_i + (\Sigma_{F_i}^{-1}\Sigma_{F_iB_i}\omega_{B_i})_i\right)\right]$$

with

$$C_i = \left(1'_{F_i}\Sigma_{F_i}^{-1}1_{F_i}\right)\left(\Sigma_{F-i}^{-1}\mu_{F_i}\right)_i + \left(1'_{F_i}\Sigma_{F_i}^{-1}mu_{F_i}\right)\left(\Sigma_{F_i}^{-1}1_{F_i}\right)_i$$

and where for convenience b_i is used for $(\omega_{F_i})i = \omega_i$, which is an asset on its bound possibly becoming free. If asset i was previously on its upper bound, $b_i = u_i$, and if it was on its lower bound, $b_i = l_i$

In order to find the maximal $\lambda^i < \lambda_c$ where an asset i formerly on its bounds currently wishes to be free, the following relation must be applied for all $i \in \mathbb{B}$

$$\lambda_{outside} = \max_{i \in \mathbb{B}}\{\lambda^i | \lambda^i < \lambda_c\}$$

Again, if no $\lambda^i < \lambda_c$, there exists no solution for $\lambda_{outside}$.

In order to find out which case will occur, the values of λ_{inside} and $\lambda_{outside}$.

- If solutions for both λ_{inside} and $\lambda_{outside}$ could be found, then the next turning point will be decided by:

$$\lambda_{new} = \max\{\lambda_{inside}, \lambda_{outside}\}$$

- If a solution is found only for either λ_{inside} or $\lambda_{outside}$, λ_{new} is overwritten by the respective value.

- Depending on which case occurs we replace $\mathbb{F}$ by $\mathbb{F}/\{i\}$ or by $\mathbb{F}_i$, $\mathbb{B}$ by $\mathbb{B} \cup \{i\}$ or $\mathbb{B}_i$ and λ_c by λ_{new}.

- If no solution for λ_{inside} and $\lambda_{outside}$ could be found, we have reached the lowest turning point and the algorithm terminates.

4.2 Treatment on the Covariance Matrix

Treating instabilities, specifically signal related instabilities, require the use of advanced mathematical techniques and a rigorous level of theory that justifies the use of these techniques. As such, there is a certain level of difficulty in resolving this issue. Yet there are plenty of techniques available to reduce instability related issues due to noise by treating the covariance to avoid ill conditioning. In this section, we will focus on a few of these methods starting with a well known technique known as the Linear Shrinkage technique.

4.2.1 The Linear Shrinkage Method

Shrinkage estimators are a class of methods that attempt to reduce the ill conditioning of a covariance matrix via reducing the condition number itself. A well known and easily implementable technique to obtain a well conditioned estimator of the covariance matrix is the Linear Shrinkage Estimator. This method can be considered as a Bayesian method as it assumes that the prior covariance matrix is the identity matrix itself. Essentially, the method attempts to shrink the covariance matrix by shifting it closer to the identity matrix.Thus the covariance matrix shifts to a form that is less prone to errors, by adjusting the condition number associated with the matrix. Many other shrinkage techniques have been introduced in the past, but we restrict ourselves to the commonly known Ledoit-Wolf linear shrinkage technique. [12]

The basic principle is straightforward, where given a correlation matrix C, we obtain a well conditioned estimator $\tilde{C}$ by obtaining the linear combination C and the identity

matrix I. The formula can be framed as follows:

$$\tilde{C} = \alpha C + (1 - \alpha)I \tag{12}$$

where $\alpha \in [0, 1]$ is the shrinkage parameter that defines the strength of shrinkage required. Total shrinkage is achieved as $\alpha \to 0$. Intuitively, when we observe at the eigen space, Λ, of the correlation matrix, the eigen values are shifted themselves towards 1. By doing this, the shrinkage estimator is essentially reducing the condition number (as the largest eigen value is reduced while the smallest eigen value is increased), thus reducing the ill conditioning of the matrix.

The choice of α depends on different considerations and various formulations have been introduced to arrive at a consistent parameter [12]. A simple approach to choosing α is by calculating the signal to noise ratio. A high signal to noise ratio would justify $\alpha = 1$, whereas a low signal to noise ratio would require cleaning and thus, a lower value of α would be required.

Unfortunately, the drawback upon using these types of estimators is that it does not attempt to segregate noise from signal. It is a long established fact that financial data has a very low signal to noise ratio. Because of the above reasons, shrinkage estimators will allow a certain reduction in noise but at the cost of an even weaker signal. To resolve this issue in particular, we look into two techniques that allows us to specifically target noisy data within the covariance matrix.

4.2.2 Eigen Value Clipping Method

The Eigen Value Clipping, or the Constant Residual Eigen Value Method, [18] is a method that involves the adjustment of noise related eigen values of a given matrix. Note that we will make use of the Marcenko-Pastur Theorem to obtain the interval of $[\lambda_-, \lambda_+]$. Consider $\{\lambda_n\}_{n=1,2,3...N}$ to be the collection of eigen values of a matrix, ordered descending, and let $i \in \mathbb{N}$ such that $\lambda_i > \lambda_+$ and $\lambda_{i+1} \leq \lambda_+$. We can perform the following transformation

on the eigen values:

$$\lambda_j = \frac{\sum_{k=i+1}^{N} \lambda_k}{N-i} \tag{13}$$

Note that the we can use any substitution to replace the eigen values associated with the noise component, as long as the trace of the denoised matrix does not change. Here, we choose to take the mean of the eigen values $\{\lambda_k\}_{k=i+1,\dots N}$, from which we can see that:

$$\sum_{j=1}^{N} \lambda_j = \sum_{j=1}^{i} \lambda_j + (N-i)\frac{\sum_{k=i+1}^{N} \lambda_k}{N-i} \tag{14}$$

We can now obtain our denoised matrix C_1 through the following transformations:

$$\tilde{C_1} = W\tilde{\Lambda}W' \tag{15}$$

$$C_1 = \tilde{C_1}\left[\left(diag\left[\tilde{C_1}\right]\right)^{\frac{1}{2}}\left(diag\left[\tilde{C_1}\right]\right)^{\frac{1}{2}'}\right]^{-1} \tag{16}$$

where $\tilde{\Lambda}$ is the diagonal matrix holding the corrected eigen values and W is the eigen vector matrix we obtain upon decomposing the original correlation matrix C to its eigen components i.e $C = W\Lambda W'$. Note that the *diag* operator is used to convert all the non diagonal elements of a given matrix. The second transformation leads to C_1 (16) is applied to rescale our initial denoised correlation matrix $\tilde{C_1}$ such that the diagonal entries become 1.

By applying the Marcenko-Pastur theorem, the method allows us to distinguish between noise related and signal related eigen values. Because the noise related eigen values would be in the lower end of the collection of eigen values and by substituting them in place of their mean, we would effectively be increasing the condition number of the matrix. We describe one more method that combines the shrinkage estimation process and the effect of the Marcenko Pastur theorem.

4.2.3 Targeted Shrinkage Method

The Targeted Shrinkage method [2] combines the use of both shrinkage techniques as well as the Marcenko-Pastur Theorem. Essentially, the shrinkage is restrained to the random eigen values alone. Consider a correlation matrix C whose eigen value decomposition can be written as:

$$C = W\Lambda W'$$

where W is the orthogonal matrix of eigen vectors while Λ is the diagonal matrix of associated eigen values. By use of the Marcenko-Pastur theorem, we can define the interval $[\lambda_-, \lambda_+]$. The targeted shrinkage technique can be used to obtain the denoised correlation matrix C_1 as follows:

$$C_1 = W_L\Lambda_L W_L' + \alpha W_R\Lambda_R W_R' + (1-\alpha)diag[W_R\Lambda_R W_R']$$

where W_R and Λ_R are the eigen values and eigen vectors associated with $\{n|\lambda_n \leq \lambda_+\}$, W_L and Λ_L are the eigen values and eigen vectors associated with $\{n|\lambda_n > \lambda_+\}$ and α is the shrinkage parameter that regulates the level of shrinkage applied on the eigen components associated with noise.

We can see how the underlying properties of the Marcenko-Pastur Theorem allows us to eliminate, or at least reduce, the noise content in the covariance matrix while keeping the signal intact. Now we move on to describing the HRP algorithm, which makes use of hierarchical relationships to assign weight allocations.

4.3 Hierarchical Risk Parity (HRP)

Although the Critical Line Algorithm was introduced specifically for inequality constrained portfolio optimization problem, there are a number of practical issues that makes CLA somewhat unreliable. A major issue is that small deviations in the predicted returns input can lead to extremely different portfolios [8]. Considering that accurate estimates of return forecasts can rarely be expected, practitioners decided to instead focus solely on the covariance matrix itself, giving rise to risk based asset allocation techniques.

But as we have seen in our previous sections, focusing on the covariance structures alone need not guarantee stable solutions. If the covariance matrix under consideration has a high condition number, i.e it is ill conditioned, then its inversion will be prone to high estimation errors. This is not ideal as the inverse of the covariance matrix is necessary when applying quadratic programming techniques to obtain optimal allocation solutions.

In the past, many techniques have been implemented to improve the robustness of optimization involving the covariance structure such as including further constraints at the cost of a smaller feasible region [15], introducing Bayesian priors [14] or, as we have seen previously, improving the stability of the inverse covariance matrix structure by shrinking the covariance matrix. The Hierarchical Portfolio Construction technique is a 3 stage process, involving Graph Theory and Machine learning, that obtains an optimal portfolio allocation, without the need for an inversion or positive definiteness of the covariance matrix.

The correlation matrix is traditionally observed as a linear algebra object that measures the cosines of the angles between vectors in the vector space formed by return series. If we were to view this as a graph, the correlation matrix can be treated as a fully connected graph where each node is an asset in the investment universe under consideration, while each edge is weighted as the correlation between the two nodes or assets [16, 17]. Algorithmically, inverting the matrix would then amount to obtaining partial correlations across the complete graph. Considering the complexity and sheer size of combinations involved in a fully connected graph (a fully connected graph G with n nodes has $\frac{n(n+1)}{2}$ edges), small estimation errors will be magnified leading to misleading results. Naturally, one can reduce this complexity by simply dropping a few edges.

What does this mean from a practical standpoint? Consider a scenario where an investor intends to create a highly diversified portfolio which includes a variety of assets such as stocks, bonds, real estate etc. The portfolio could be further enriched by considering different classes of assets within a given asset. For example, the stocks in the portfolio may be grouped based on size, liquidity, industry or any other indicator. When

attempting to substitute any asset from the current portfolio, we naturally expect the substitution to be made with an asset that has similar characteristics. If the investor wishes to add or reduce the allocation to a large publicly traded U.S. financial stocks like J. P. Morgan, it is more reasonable to consider reducing or adding the allocation to another large publicly traded U.S. bank like Goldman Sachs, rather than a small community bank in Switzerland. Yet the correlation matrix does not behave with this notion. Rather any asset in the considered investment universe is considered a good substitute for another and the only key determiner of the allocation is the correlation between these assets. In other words, the correlation matrix lacks the notion of *hierarchy*, and because of this, weights can move freely among the assets allowing for instability.

The HRP technique [5, 6] visualizes the correlation matrix as a tree structure, that has the notion of hierarchy, rather than a fully connected graph. This introduces two benefits to portfolio allocation, namely:

1. A tree has only $N - 1$ edges for N nodes, which is a significant reduction in complexity as compared to a fully connected graph. It allows the weights to rebalance amongst its peers at a given hierarchy rather than with all other assets under consideration.

2. The weights can be distributed top down, which is useful as many asset managers proceed with portfolio allocation in a similar approach i.e from asset class to sectors to individual securities.

The HRP algorithm can be divided into three stages: Tree Clustering, Quasi Diagonalization and Recursive bisection. We take a look into each of the stages below.

4.3.1 Stage 1: Tree Clustering

The goal of this stage is to create a hierarchical collection of clusters from the assets under consideration, such that we can assign the nodes to each level of the tree structure. Let us consider the same data set as described in Section 2 i.e the returns of N assets in T time periods. Let us also obtain the correlation matrix C, whose entries are defined as $C = \{\rho_{ij}\}_{i,j=1,2,\ldots,N}$. Note that $\rho_{ij} = \rho\left[X_i, X_j\right]$.

1. Define a distance measure $d : (X_i, X_j) \subset B \rightarrow \mathbb{R} \in [0, 1]$ such that

$$d_{ij} = d\left[X_i, X_j\right] = \sqrt{\frac{(1 - \rho_{ij})}{2}}$$

where $B = \{(X_i, X_j) : \forall i = 1, 2, \ldots N, j = 1, 2, \ldots N\}$. Using d, we can create an $N \times N$ distance matrix $D = \{d_{ij}\}_{i,j=1,2,\ldots N}$. This matrix D is a proper metric space. In other words, it satisfies the following properties:

(a) $d(x, y) \geq 0$

(b) $d(x, y) = 0 \Leftrightarrow x = y$

(c) $d(x, y) = d(y, x)$

(d) $d(x, z) \leq d(x, y) + d(y, z)$

2. Now that we have created a distance matrix consisting of distances between any two coloumn vectors of X, we create a new distance measure $\tilde{d} : (D_i, D_j) \rightarrow \mathbb{R} \in \left[0, \sqrt{N}\right]$ such that:

$$\tilde{d}_{ij} = \tilde{d}\left[D_i, D_j\right] = \sqrt{\sum_{n=1}^{N} (d_{ni} - d_{nj})^2}$$

We are essentially calculating the Euclidean distance between two coloumn vectors of D. Note that $\tilde{d}$ is a distance measure defined over the the metric space D as opposed to X and thus a function of the correlation matrix.

3. Cluster together pairs of columns (i^*, j^*) based on the rule

$$arg\left(\min_{i,j,i \neq j} \tilde{d}_{ij}\right)$$

Denote this cluster as $u\left[1\right]$.

4. Once we have formed our clusters, the next step is to define the distance $\bar{d}$ between the newly formed cluster $u\left[1\right]$ and the unclustered items. This distance will be used

to update the distance matrix $\tilde{D}$. We can define the distance $\bar{d}$ as:

$$\bar{d}_{i,u[1]} = \min\{\tilde{d}_{ij}\}_{j \in u[1]}$$

5. Once we have obtained the distance measure $\bar{d}$, we append it as a row and coloumn to the distance matrix $\tilde{D}$, after which we remove any columns or rows associated with $j \in u\,[1]$.

6. Repeat steps 3,4 and 5 to obtain and append $N - 1$ clusters. Note that the final cluster will contain all original entries, a stopping point for the clustering algorithm.

Now that we have completed the clustering our entries into a hierarchical structure, we move on to the next stage.

4.3.2 Stage 2: Quasi Diagonalization

This stage involves reorganizing the covariance matrix structure, such that the largest values lie on the diagonal, resulting in a quasi diagonal matrix. A block diagonal matrix is a square diagonal matrix where the diagonal elements are square matrices of any size, whereas the off diagonal elements are 0. A matrix A is said to be quasi diagonalizable if we can transform it into a block diagonal matrix with the diagonal matrices having atmost an order of 2.

Quasi Diagonalization of the covariance matrix brings out a useful property wherein similar investments are placed together while dissimilar investments are held away from each other. The end result is a block diagonal matrix where the prominent values lie across the diagonal blocks, a structure similar to a diagonal matrix but with matrices as the elements. The algorithm is basically replacing each cluster from the linkage matrix with its constituents. Since the clusters are sorted hierarchically, we eventually obtain the unclustered items with the sorting preserved.

4.3.3 Stage 3: Recursive Bisection

Now that we have transformed our covariance matrix to a quasi diagonalizable form, we move on to the last stage of the HRP algorithm. As the name suggests, we will be recursively bisecting the list of assets under consideration while assigning a weight at each step of the recursive process. Note that the inverse allocation is optimal for a diagonal matrix. We will discuss the algorithm and discuss each step in further detail The following algorithm formalizes the idea:

1. Set the list of items as $L = \{L_0\}$, with $L_0 = \{n\}_{n=1,2,\dots N}$, and assign a unit weight to all items i.e $w_n = 1$, $\forall n = 1, 2, \dots N$.

2. If $|L_i| = 1$, $\forall L_i \in L$, then stop.

3. For each $L_i \in L$, where $|L_i| > 1$,

   (a) Bisect L_i as $L_i = L_i^{(1)} \bigcup L_i^{(2)}$, where $|L_i^{(1)}| = int\left[\frac{1}{2}|L_i|\right]$.

   (b) Define the variance of $L^(j)_i, j = 1, 2$ as the quadratic form $\tilde{V}_i^{(j)} = \tilde{w}_i^{(j)\prime} V_i^j \tilde{w}_i^j$, where V_i^j is the covariance matrix between the partitions of L_i, and $w_i^{(j)} = \frac{diag\left[V_i^{(j)}\right]^{-1}}{tr\left[diag\left[V_i^{(j)}\right]^{-1}\right]}$.

   (c) Compute the split factor: $\alpha_i = 1 - \frac{\tilde{V}_i^{(1)}}{\tilde{V}_i^{(1)} + \tilde{V}_i^{(2)}}$, so that $0 \leq \alpha_i \leq 1$

   (d) Rescale allocations w_n by a factor of α_i, $\forall n \in L_i^{(1)}$.

   (e) Rescale allocations w_n by a factor of $(1 - \alpha_i)$, $\forall n \in L_i^{(2)}$.

4. Loop to step 2.

Our first step is to initialize the algorithm with the list L. We use a list to preserve the hierarchical order obtained in the previous stages. Initially the list contains $L_0 \in L$, where L_0 contains all the assets under consideration, but at each step we will obtain two lists for each list contained in L with close to half the number of elements. This will proceed until all the lists in L contain only one asset, and the union of all elements in L in the end must naturally contain the n assets under consideration.

The next step is to assign variances. Since we are working with a quasi diagonal matrix, and including the fact that inverse allocations are optimal for diagonal elements, we will be updating the weights w_n at each stage of the recursion. Assigning $w_n = 1, \forall n = 1, 2, \ldots N$ and partitioning a list L_i into $L_i^{(1)}$ and $L_i^{(2)}$, we obtain the covariance matrix V_i^j between these two partitions, from which we obtain the weights based on the inverse allocation scheme. Essentially we are assigning weights to each list as a whole (bottom up approach). The final step is to assign variances to each constituent of the partitioned list, by way of the splitting factor α_i (top-down approach). Proceeding in this manner recursively, we can see how the inverse allocation weighting scheme is brought out in a hierarchical form. The algorithm guarantees that $w_i \in [0, 1], \forall i = 1, 2, \ldots N$ and that $\sum_{i=1}^{N} w_i = 1$ since at each stage, we are splitting the weights received from the higher hierarchical levels. Constraints can be added according to user preference by replacing the equations involving the splitting factor (α_i) accordingly (in steps 3c,3d and 3e). This concludes the process of the Hierarchical Risk Parity technique.

Having discussed the HRP technique, we can see how the algorithm avoids the Markowitz curse by not involving the inverse covariance matrix at any stage. By introducing the hierarchical assignment to the assets (in Stage 1 and Stage 2) the covariance matrices of the lists (V_i^j) would be better conditioned to work on. We now move on to one more technique that uses the notion of clustering to produce stable results, known as the Nested Clustered Optimization Method.

4.4 Nested Clustered Optimization Method (NCO)

The Nested Cluster Optimization technique is a technique introduced solely to deal with the instability caused by signal structure (discussed in Section 3.2) [1, 2]. At its core, the NCO algorithm attempts to control the instability within clusters obtained from the covariance matrix. Before we move on to explaining the individual steps in the algorithm, we describe the intuition behind this approach below.

When the correlation matrix is an identity matrix, the associated eigen value function can be viewed as a horizontal line, which can attributed to the lack of correlation. Natu-

rally, the condition number would be 1 in this specific instance. As soon as we introduce a non zero correlation into the correlation matrix, the eigen value function rotates from the previous case, and irregular clusters are introduced. As such, the condition number would also increase.

Financial securities, in particular, would have a high presence of clusters within the correlation matrix due to the presence of hierarchical relationships (and the resulting correlations) between securities. Consider the case of K securities which form a cluster together. These securities are more exposed to a common eigen vector, implying that the associated eigen value will explain a greater part of their variance. But as we have seen before in Section 3.2, this increase in the eigen value due to the relationships between the securities will lead to a decrease in the remaining eigen values of that cluster, restricted by the trace of the matrix, which will naturally lead to a higher condition number. In short, the greater the intracluster correlations (i.e correlation within the clusters), the greater the condition number.

The NCO attempts to contain this instability from affecting the entire asset universe by focusing on optimizing each cluster individually. One can argue that optimizing the cluster causing the high condition number (dominant cluster) may be enough. But by ignoring the remaining clusters, extreme eigen values would still exist and will cause instabilities to affect the whole matrix.

Before we move on to describing the NCO process, it is important to note that the instability this algorithm is attempting to resolve is unrelated to the noise induced instability. As such it is important to denoise the covariance matrix before we move ahead. Now we explain the procedure below:

1. **Cluster Formation** The first step of the NCO is to form the clusters of the correlation matrix. This can be performed using any clustering algorithm.

2. **Intracluster Weights** Obtain the optimal intracluster allocation weights for each of the clusters. With this we can form the reduced covariance matrix that describes the correlations within each cluster.

3. **Intercluster weights** We finally obtain the optimal intercluster weights using the reduced covariance matrix from the previous step. By performing the previous steps of clustering and intracluster weight optimization, we will obtain a matrix close to the diagonal matrix. In this way, the NCO method mitigates the issues that arise because of the Markowitz curse.

4. **Calculating Final Weights** We can finally obtain the final optimal weight allocation by performing the dot product of the intracluster and intercluster weights.

Note that the optimal intercluster and intracluster weights can be obtained through any standard optimization process. The NCO algorithm is agnostic regarding this choice.

5 Monte Carlo Optimization Selection (MCOS)

Now that we have discussed a few methods that attempts to provide a more robust allocation decision, the next obvious step is to compare the performance of these methods, specifically their ability to provide stable solutions given a set of inputs. We make use of a Monte Carlo based experiment to achieve this, which is of topic of focus in this section.

A Monte Carlo experiment is essentially a class of algorithms that is useful to solve problems through the use of randomness, by means of repeated random samplings from a given distribution. A historical backtest makes use of historical data to make a judgement on a strategy. This would become an issue as comparing the performance of a method based on historical data allows for a variety of biases. In contrast, given a large number of simulations, the Monte Carlo approach is useful in gaining an accurate interpretation of the given methods performance. The randomness in sampling allows us to test the effectiveness of a technique in a variety of simulated conditions.

In the past, a large variety of methods have been introduced to solve instability issues in different problems. It is unreasonable to expect that any one method from these will be effective in controlling instability in all circumstance. A lot of these methods are effective in their goals when subjected to a particular type of constraint. For instance, many of the methods discussed here may not be effective when applied outside the field of finance.

Thus it is important that we are able to compare different methods for a given set of inputs with as little bias as possible.

It is here where the Monte Carlo Optimization Selection method (MCOS) comes into play [1, 2]. The MCOS method is introduced for the sole purpose of estimating the allocation error produced by various methods given a set of input variables, and is thus especially useful to decide which method performs better for a specific set of inputs.

We will first discuss the implementation of the MCOS method to obtain the allocation estimate $\tilde{\omega}^*$. For our purpose, the inputs for the method include the vector of expected returns, μ, and the covariance matrix, Σ. The procedure is as follows:

1. **Draw Empirical Means and Covariance matrices:**

   From the true dataset X (dimensions $T \times N$), we obtain the true values of our inputs, (μ, Σ). We then proceed to simulate data $\tilde{X}_1$, with the same dimensions as X and based on the distribution characterized by (μ, Σ), after which we generate the simulated pair $\left(\tilde{\mu}_1, \tilde{\Sigma}_1\right)$.

2. **Denoise the Covariance Matrix:**

   In this step, we treat the noise present in the covariance matrix to avoid noise related instability issues, as discussed in Section 3.1. Note that this step need not be solely focused on denoising and can be used to treat the inputs in any manner that seems beneficial to the optimization process, such as the Ledoit- Wolf shrinkage technique.

3. **Obtaining Optimal Allocations:**

   Now we obtain the optimal allocation estimate $\tilde{\omega}_1^*$ from the simulated pair $\left(\tilde{\mu}_1, \tilde{\Sigma}_1\right)$, using the methods under investigation. The MCOS algorithm is agnostic as to the type of method used to obtain $\tilde{\omega}_1^*$ and is therefore, flexible in judging the performance of a variety of methods.

4. **Performing Monte Carlo Simulations:**

   At this stage, we repeat the steps outlined previously in the form of a Monte Carlo

experiment, wherein we obtain optimal allocations $\{\tilde{\omega}_i^*\}_{i=1,2,\dots N}$, for a collection of simulated pairs , $\left\{\left(\tilde{\mu}_i, \tilde{\Sigma}_i\right)\right\}_{i=1,2,\dots N}$.

5. **Calculating the Estimation Errors:**

   After obtaining the optimal allocations $\{\tilde{\omega}_i^*\}_{i=1,2,\dots N}$ from the simulated inputs, we now obtain the true optimal allocation ω^* from the original true inputs (μ, Σ). At this stage, we are now capable of calculating and comparing the estimation errors between $\{\tilde{\omega}_i^*\}_{i=1,2,\dots N}$ and ω^*.

The estimation error can be calculated by obtaining the standard deviations of the differences between the optimal allocations ω^* and $\tilde{\omega}_i^*$ for the given variable (μ, Σ). When performed upon multiple variables (μ_k, Σ_k), where $k = 1, 2, \dots K$, the mean of the standard deviations, σ_k, can be used to provide a general comparison the methods under investigation. Note that when working on actual data (as opposed to randomly generated data), one can compare the methods specifically for that data. This is where the MCOS becomes useful, as it allows users to compare the stability of different methods for a specific data set rather than in general. Other alternatives to calculate estimation error can also be implemented such as:

- The mean difference in expected outcomes $\left(\omega^* - \tilde{\omega_i}^*\right)' \mu$
- The mean difference in variance: $\left(\omega^* - \tilde{\omega_i}^*\right)' \Sigma \left(\omega^* - \tilde{\omega_i}^*\right)$
- the mean difference in Sharpe Ratio: $\dfrac{\left(\omega^* - \tilde{\omega_i}^*\right)' \mu}{\sqrt{\left(\omega^* - \tilde{\omega_i}^*\right)' \Sigma \left(\omega^* - \tilde{\omega_i}^*\right)}}$

Now that we have discussed the MCOS algorithm and its implementation process, we can move on to the performing the numerical tests to compare the methods that control for instability.

6 Numerical Experiments

This section is dedicated to testing out the efficiency of the methods discussed in this report as compared to a base technique with respect to their ability to provide stable

solutions. We will make use of the MCOS method discussed in Section 5 to perform the comparison, which would mean that the inputs would be simulated over a number of runs for testing.

Our first comparison will be on the methods that treat the covariance method i.e Ledoit-Wolf Shrinkage, Eigen Clipping and Targeted Shrinkage techniques. We will perform the standard optimization via the *quadprog* solver in Python, with and without the application of these methods, with which we will define the effective noise reducing technique.

After selecting the adequate noise reduction technique, we will move on to comparing the 3 techniques described above, namely the CLA algorithm, the HRP algorithm and the NCO algorithm. We also include the results from optimizing the portfolio through the inbuilt *quadprog* solver in Python. Note that the NCO algorithm will make use of the *quadprog* solver to obtain the intercluster and intracluster weights. The comparison for these methods will be performed as per the steps in the MCOS algorithm.

Finally, we test the above optimization techniques on the returns of a collection of 50 assets and check the performance of the portfolios. With this we will conclude the testing of the various techniques. The code for these tests are presented in the appendix section.

6.1 Test 1: Comparison of Denoising techniques

In this test, our goal is to perform the standard constrained optimization via the *quadprog* solver in Python, with and without the denoising techniques and compare their performance by observing the standard deviation of the RMSE values of each allocation. We further compare the Targeted shrinkage method under various cases of the shrinkage parameter ($\alpha = 0.1, 0.5$ and 0.9). The Ledoit- Wolf shrinkage is applied as well on all the above cases to check for any significant improvement in results.

The $formTrueMatrix()$ computes the expected returns and covariance matrix out of 10 blocks of size 10 each, with off diagonal elements having a correlation of 0.5. The variances are drawn from a uniform distribution bounded between 5% and 20% while the mean vector is drawn from a normal distribution with mean and standard deviation

equal to the standard deviation from the covariance matrix.

$DeNoiseTest()$ first obtains the mean vector and covariance matrix estimates from the simulation of 10000 observations, using the $simCovMu()$ function. Then we apply the $deNoiseCov()$ function to obtain the new covariance matrices after performing the shrinkage and denoising techniques. We finally use the solver to estimate the optimal allocations for the minimum variance portfolio, using the $optPortcon()$ function, under each scenario and compute the standard deviation of the RMSE values in each observation. The results are described in the Table below: Based on the results in Table 1, we

Table 1: Denoising techniques comparison based on standard deviation of RMSE ($\times 10^{-3}$)- Minimum Variance Portfolio

	No Denoising	EC	TS(α=0.1)	TS(α=0.5)	TS(α=0.9)
No Shrinkage	1.581	1.393	1.569	1.665	1.980
Shrinkage	1.615	1.423	1.574	1.549	1.765

can see that the Eigen Value Clipping method proves to show the least variation in errors given different samples of inputs. The Targeted shrinkage technique on the other hand shows varying results with varying levels of the shrinkage parameter α. For $\alpha = 0.1$, the results are better than without the application of denoising, while for the remaining two cases, the results prove much worse. In the end, the Eigen clipping method proves to be preferable among the two.

Applying shrinkage yields relatively worse results overall, except in when applying Targeted shrinkage at $\alpha = 0.5$ and $\alpha = 0.9$. This clearly shows how the shrinking may affect the signal as well as the noise, and its benefits may rely solely on the strength of the denoising technique.

We now move on to testing these methods again for the Maximum Sharpe ratio allocations and observe any differing results.

The results seem similar to the previous case, where the Targeted shrinkage method performs sufficiently well when $\alpha = 0.1$ while it is worse for when $\alpha = 0.5$ and $\alpha = 0.9$, when

Table 2: Denoising techniques comparison based on standard deviation of RMSE ($\times 10^{-3}$)-Maximum Sharpe Ratio Portfolio

	No Denoising	EC	TS(α=0.1)	TS(α=0.5)	TS(α=0.9)
No Shrinkage	1.950	1.679	1.930	2.043	2.428
Shrinkage	1.962	1.706	1.939	2.045	2.427

compared with the allocations obtained with no denoising. The Eigen clipping method still dominates the other methods in providing least variation of the RMSE, while shrinkage provides little to no benefits at all.

We now move on to testing the optimization techniques with simulated data using the MCOS method. Note that we will be using the Eigen clipping method to denoise the covariance matrix due to its relatively good performance and unambiquity in terms of parameter selection (as is the case for Targeted shrinkage with respect to α).

6.2 Test 2: Comparison of Optimization Techniques on Simulated Data

We make use of the covariance and mean inputs generated before to simulate 1000 observations and obtain the minimum variance portfolio allocations using the *quadprog* solve, the CLA method, the HRP method and the NCO method. We make use of the MCOS methodology described before, with which we obtain measures for variation of the RMSE of the allocations and variation of the expected returns, with respect to the base method CLA. The *monteCarlo*() function applies this methodology to obtain the results below. As previously mentioned, the Eigen clipping technique is applied on the covariance matrix for more robust results. The table below lists our findings: The results above indicate that the NCO method shows least variation of results with respect to the CLA allocations, both in terms of RMSE of the allocations and the difference between returns. The HRP algorithm shows favourable results for RMSE while has relatively poor scores with respect to the difference in returns. This would most likely be due to the fact that it is

Table 3: Comparison of Optimization techniques on Simulated Data based on standard deviation of RMSE and Return difference ($\times 10^{-3}$)

	MVO	CLA	HRP	NCO
RMSE	1.407	1.553	1.361	0.794
Return	1.11	2.49	3.46	0.59

a risk parity portfolio, thus leading to lower returns.

The CLA method shows higher variation as compared to the inbuilt quadratic programming solver, indicating that although theoretically it is expected to provide exact results of the optimal weights in the constrained case, it is highly subject to input errors.

We now move on to the final test, where we apply these methods to actual data and check their overall performance.

6.3 Test 3: Comparison of Optimization techniques on Actual Data

In this test, we have obtained the mean and covariance inputs from the monthly return data, from 2000-2020, of 50 assets listed in the $SP100$ index in varying sectors. The inputs are obtained for each month between 2005-2020 from the return data to the prior 5 years. By applying the $DataTestOpt()$ function, we obtain the monthly allocations for each month during 2005-2020 using each of the optimization methods listed above. The performance metrics used for comparison are the overall portfolio return, the portfolio volatility and the associated Sharpe Ratio. We also calculate a turnover measure i.e the Portfolio Turnover ratio, which is the average change in the weight allocations of each of these methods during the given time period. It is computed as the average of the absolute deviation of weights between each month, assuming that rebalancing occurs monthly. The table below shows the results. We can see from Table 4 that the expected returns are higher for the HRP algorithm, following with the NCO algorithm. Yet the standard

Table 4: Comparison of Optimization techniques on Actual Data

	MVO	CLA	HRP	NCO
Return	0.9936×10^{-2}	1.0189×10^{-2}	1.2195×10^{-2}	v1.1210 $\times 10^{-2}$
Standard Deviation	2.685×10^{-3}	2.717×10^{-3}	4.460×10^{-3}	5.228×10^{-3}
Sharpe Ratio	3.70	3.75	2.73	2.14
Turnover Ratio	0.152	0.17	0.52	1.18

deviation of the return series indicates a lower volatility for the standard quadratic programming approach, followed by the CLA method. The Sharpe ratio of the portfolios also indicates better performance for the CLA method and *quadprog* solver, as compared to the NCO and HRP methods. When comparing the Turnover ratios, we see that there is higher turnover for the NCO and HRP portfolio allocations.

Overall, these performance measures indicate that the NCO and HRP techniques do not perform as expected with actual data as compared to the simulated data. A potential reason for this is the difficulty in clustering the covariance matrix adequately with the presence of market related principal components. Since the market tends to have some level of correlation with other factors, appropriate treatment to remove this effect will be required for the optimal allocations to obtained with ease.

7 Conclusion

From the results of our numerical tests, we see that the clustering applications of the NCO and HRP techniques work reasonably well on simulated data that resembles an ideal covariance matrix of the returns of various assets. Yet when tested on the actual data, we observe that it underperforms in terms of Sharpe Ratio and, most importantly, the turnover measure, indicating a less robust allocation.

Part of the reason for this would be due to the presence of factors that has an effect on all the assets (such as the market component) leading to greater difficulty in clustering

the covariance matrix adequately which in turn leads to improper allocation of assets. Further treatment of the covariance matrix may be required to fully utilize the benefits of the NCO and HRP techniques in obtaining robust optimal allocations.

References

[1] Lopez de Prado, Marcos, A Robust Estimator of the Efficient Frontier (October 15, 2016). Available at SSRN: https://ssrn.com/abstract=3469961 or http://dx.doi.org/10.2139/ssrn.3469961

[2] Lopez de Prado, M. (2019): Machine Learning for Asset Managers. Cambridge University Press, First edition

[3] Markowitz, H. M. (1952): Portfolio selection. Journal of Finance, 7(1), pp. 7791.

[4] Bailey, D. and M. Lopez de Prado (2013): An open-source implementation of the critical-line algorithm for portfolio optimization. Algorithms, 6(1), pp. 169-196. http://www.mdpi.com/1999-4893/6/1

[5] Lopez de Prado, M. (2018): Advances in financial machine learning. Wiley, First edition.

[6] Lopez de Prado, M. (2016): Building diversified portfolios that outperform out-of-sample. The Journal of Portfolio Management, 42(4), pp. 59-69.

[7] De Miguel, V., L. Garlappi, and R. Uppal (2009): Optimal versus nave diversification: how inefficient is the 1/N portfolio strategy? Review of Financial Studies, 22(5), pp. 1915-1953.

[8] Michaud, R. (1998): Efficient asset allocation: a practical guide to stock portfolio optimization and asset allocation. Boston, MA: Harvard Business School Press.

[9] Jurczenko, E. Risk-Based and Factor Investing. Elsevier Science (2015).

[10] Guerard, J. (2010): Handbook of portfolio construction. Springer, First edition.

[11] Kuhn, H. W. and A. W. Tucker. "Nonlinear programming". Proceedings of 2nd Berkeley Symposium. Berkeley: University of California Press (1952). pp. 481492.

[12] Ledoit, O. and M. Wolf (2003): A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices. Journal of Multivariate Analysis, 88(2), pp. 365-411.

[13] Marenko, V. and L. Pastur (1967): Distribution of eigenvalues for some sets of random matrices. Matematicheskii Sbornik, Vol. 72, No. 4, pp. 507-536.

[14] Black, F. and R. Litterman. Global portfolio optimization. Financial Analysts Journal, Vol. 48 (1992), pp. 2843.

[15] Clarke, R., H. De Silva, and S. Thorley. Portfolio constraints and the fundamental law of active management. Financial Analysts Journal, Vol. 58 (2002), pp. 4866.

[16] Calkin, N. and M. Lpez de Prado. Stochastic Flow Diagrams. Algorithmic Finance, Vol. 3, No. 1 (2014), pp. 21-42. Availble at http://ssrn.com/abstract=2379314

[17] Calkin, N. and M. Lpez de Prado. The Topology of Macro Financial Flows: An Application of Stochastic Flow Diagrams. Algorithmic Finance, Vol. 3, No. 1 (2014), pp. 43-85. Available at http://ssrn.com/abstract=2379319

[18] Potter, M., J.P. Bouchaud, L. Laloux (2005): Financial applications of random matrix theory: Old laces and new pieces. Acta Physica Polonica B, Vol. 36, No. 9, pp. 2767-2784.

[19] Bailey, D. and M. Lopez de Prado (2013): An open-source implementation of the critical-line algorithm for portfolio optimization. Algorithms, 6(1), pp. 169-196. http://www.mdpi.com/1999-4893/6/1

[20] Niedermayer, A.; Niedermayer, D. Applying Markowitzs Critical Line Algorithm. Research Paper Series, Department of Economics, University of Bern, Bern, Switzerland, 2007.

A Appendix

Listing 1: Insert code directly in your document

```
#Importing necessary packages
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import pandas_datareader.data as web
from scipy.optimize import minimize
from scipy.linalg import block_diag
import scipy.cluster.hierarchy as sch
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KernelDensity
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples
from sklearn.covariance import LedoitWolf
import datetime as datetime
import quadprog
import random



#
#Functions with description
#

def quadprog_solve_qp(P, q, A=None, b=None, G=None, h=None):
    qp_G = .5 * (P + P.T)   # make sure P is symmetric
    qp_a = -q
    if G is not None:
        qp_C = -np.vstack([A, G]).T
        qp_b = -np.hstack([b, h])
    else:
        qp_C = -A.T
        qp_b = -b
    meq=1
```

```
    return quadprog.solve_qp(qp_G, qp_a, qp_C, qp_b, meq)[0]

def optPortcon(cov,mu=None):
    if mu is not None:
        cov=np.array(cov)
        mu=np.array(mu)
        mu0=np.zeros(cov.shape[0])
        A=mu.reshape((1,cov.shape[0]))
        b=np.ones((1,))
        G=np.diag( np.ones(cov.shape[0]))
        h=np.zeros(cov.shape[0])
        wgt=quadprog_solve_qp(cov, mu0, A=A, b=b, G=G, h=h)
        wgt=wgt/wgt.sum()
    else:
        cov=np.array(cov)
        mu=np.zeros(cov.shape[0])
        A=np.ones((1,cov.shape[0]))
        b=np.ones((1,))
        G=np.diag( np.ones(cov.shape[0]))
        h=np.zeros(cov.shape[0])
        wgt=quadprog_solve_qp(cov, mu, A=A, b=b, G=G, h=h)
        wgt=wgt/wgt.sum()
    wgt=wgt.reshape((wgt.shape[0],1))
    return wgt

#Generates mean and covariance inputs for artificial data testing
def formTrueMatrix(nBlocks,bSize,bCorr):
  #forms block diagonal matrix
  #bCorr  correlation in diagonal blocks
  corr0=formBlockMatrix(nBlocks,bSize,bCorr)
  corr0=pd.DataFrame(corr0)
  #shuffling coloumns list
  cols=corr0.columns.tolist()
  np.random.shuffle(cols)
  #new correlation matrix with shuffled coloumns
```

```
    #i.e block diagonal format lost
    corr0=corr0[cols].loc[cols].copy(deep=True)
    #assigning random standard deviation between 5% and 20%
    std0=np.random.uniform(.05,.2,corr0.shape[0])
    #creating covariance matrix
    cov0=corr2cov(corr0,std0)

    mu0=np.random.normal(std0,std0,cov0.shape[0]).reshape(1,1)
    return mu0,cov0

#Forms block matrix for testing on artificial data
def formBlockMatrix(nBlocks,bSize,bCorr):
    #creates a matrix with elements as bCorr
    block=np.ones((bSize,bSize))*bCorr
    #assigns 1 to diagonal elements
    block[range(bSize),range(bSize)]=1
    #creates block diagonal matrix
    #diagonal blocks same as block variable
    #off diagonal blocks are 0 matrix
    corr=block_diag(*([block]*nBlocks))
    return corr

#Gets covariance matrix from correlation matrix and standard deviations
def corr2cov(corr,std):
    #calculating covariance matrix
    cov=corr*np.outer(std,std)
    return cov

#Gets correlation matrix from covariance matrix
def cov2corr(cov):
    # obtaining standard deviations
    std=np.sqrt(np.diag(cov))
    #calculating correlation matrix
    corr=cov/np.outer(std,std)
    #changing unrealistic correlation values
```

```
    corr[corr<1],corr[corr>1]=1,1
    return corr


def DenoiseTest(mu0,cov0,nObs,nTrials,minVarPortf):
    for i in range(nTrials):
        mu1,cov1=simCovMu(mu0,cov0,nObs,shrink=False)
        mu1s,cov1s=simCovMu(mu0,cov0,nObs,shrink=True)
        if minVarPortf:
            mu1=None
            mu1s=None
        cov1_EC,cov1_TS,cov2_TS,cov3_TS=deNoiseCov(cov1,nObs*1./cov1.shape[1])
        cov1s_EC,cov1s_TS,cov2s_TS,cov3s_TS=deNoiseCov(cov1s,nObs*1./cov1s.shape[1])

        w1.loc[i]=optPortcon(cov1,mu1).flatten()
        w1_EC.loc[i]=optPortcon(cov1_EC,mu1).flatten()
        w1_TS.loc[i]=optPortcon(cov1_TS,mu1).flatten()
        w1_TS2.loc[i]=optPortcon(cov2_TS,mu1).flatten()
        w1_TS3.loc[i]=optPortcon(cov3_TS,mu1).flatten()



        w1s.loc[i]=optPortcon(cov1s,mu1s).flatten()
        w1s_EC.loc[i]=optPortcon(cov1s_EC,mu1s).flatten()
        w1s_TS.loc[i]=optPortcon(cov1s_TS,mu1s).flatten()
        w1s_TS2.loc[i]=optPortcon(cov2s_TS,mu1s).flatten()
        w1s_TS3.loc[i]=optPortcon(cov3s_TS,mu1s).flatten()

        print("T1_Iteration:_"+str(i)) #Test
    return w1,w1_EC,w1_TS,w1_TS2,w1_TS3,w1s,w1s_EC,w1s_TS,w1s_TS2,w1s_TS3

#CLA function
class CLA_3:
    def __init__(self,mean,covar,lB,uB):
        # Initialize the class
        self.mean=mean
```

```
        self.covar=covar
        self.lB=lB
        self.uB=uB
        self.w=[] # solution
        self.l=[] # lambdas
        self.g=[] # gammas
        self.f=[] # free weights
#
    def solve(self):
        # Compute the turning points,free sets and weights
        f,w=self.initAlgo()
        self.w.append(np.copy(w)) # store solution
        self.l.append(None)
        self.g.append(None)
        self.f.append(f[:])
        while True:
            #1) case a): Bound one free weight
            l_in=None
            if len(f)>1:
                covarF,covarFB,meanF,wB=self.getMatrices(f)
                covarF_inv=np.linalg.inv(covarF)
                j=0
                for i in f:
                    l,bi=self.computeLambda(covarF_inv,covarFB,meanF,wB,j, \
[self.lB[i],self.uB[i]])
                    if (l_in is None or l>l_in):l_in,i_in,bi_in=l,i,bi
                    j+=1
            #2) case b): Free one bounded weight
            l_out=None
            if len(f)<self.mean.shape[0]:
                b=self.getB(f)
                for i in b:
                    covarF,covarFB,meanF,wB=self.getMatrices(f+[i])
                    covarF_inv=np.linalg.inv(covarF)
                    l,bi=self.computeLambda(covarF_inv,covarFB,meanF,wB, \
```

```
meanF.shape[0] 1,self.w[ 1][i])
                    if (self.l[1]==None or l<self.l[ 1]) \
and (l_out is None or l>l_out):
                l_out,i_out=l,i
            if (l_in==None or l_in<0) and (l_out==None or l_out<0):
                #3) compute minimum variance solution
                self.l.append(0)
                covarF,covarFB,meanF,wB=self.getMatrices(f)
                covarF_inv=np.linalg.inv(covarF)
                meanF=np.zeros(meanF.shape)
            else:
                #4) decide lambda
                if l_out is None or (l_in is not None out l_in>l_out):
                    self.l.append(l_in)
                    f.remove(i_in)
                    w[i_in]=bi_in # set value at the correct boundary
                else:
                    self.l.append(l_out)
                    f.append(i_out)
                covarF,covarFB,meanF,wB=self.getMatrices(f)
                covarF_inv=np.linalg.inv(covarF)
            #5) compute solution vector
            wF,g=self.computeW(covarF_inv,covarFB,meanF,wB)
            for i in range(len(f)):w[f[i]]=wF[i]
            self.w.append(np.copy(w)) # store solution
            self.g.append(g)
            self.f.append(f[:])
            if self.l[1]==0:break
        #6) Purge turning points
        self.purgeNumErr(10e 10)
        self.purgeExcess()
#
    def initAlgo(self):
        # Initialize the algo
        #1) Form structured array
```

```
        a=np.zeros((self.mean.shape[0]),dtype=[('id',int),('mu',float)])
        b=[self.mean[i][0] for i in range(self.mean.shape[0])] # dump array into list
        a[:]=list(zip(range(self.mean.shape[0]),b)) # fill structured array
        #2) Sort structured array
        b=np.sort(a,order='mu')
        #3) First free weight
        i,w=b.shape[0],np.copy(self.lB)
        while sum(w)<1:
            i=1
            w[b[i][0]]=self.uB[b[i][0]]
        w[b[i][0]]+=1 sum(w)
        return [b[i][0]],w
#
    def computeBi(self,c,bi):
        if c>0:
            bi=bi[1][0]
        if c<0:
            bi=bi[0][0]
        return bi
#
    def computeW(self,covarF_inv,covarFB,meanF,wB):
        #1) compute gamma
        onesF=np.ones(meanF.shape)
        g1=np.dot(np.dot(onesF.T,covarF_inv),meanF)
        g2=np.dot(np.dot(onesF.T,covarF_inv),onesF)
        if wB is None:
            g,w1=float(self.l[1]*g1/g2+1/g2),0
        else:
            onesB=np.ones(wB.shape)
            g3=np.dot(onesB.T,wB)
            g4=np.dot(covarF_inv,covarFB)
            w1=np.dot(g4,wB)
            g4=np.dot(onesF.T,w1)
            g=float(self.l[1]*g1/g2+(1 g3+g4)/g2)
        #2) compute weights
```

```
        w2=np.dot(covarF_inv,onesF)
        w3=np.dot(covarF_inv,meanF)
        return  w1+g*w2+self.l[1]*w3,g
#
    def computeLambda(self,covarF_inv,covarFB,meanF,wB,i,bi):
        #1) C
        onesF=np.ones(meanF.shape)
        c1=np.dot(np.dot(onesF.T,covarF_inv),onesF)
        c2=np.dot(covarF_inv,meanF)
        c3=np.dot(np.dot(onesF.T,covarF_inv),meanF)
        c4=np.dot(covarF_inv,onesF)
        c=c1*c2[i]+c3*c4[i]
        if c==0:return
        #2) bi
        if type(bi)==list:bi=self.computeBi(c,bi)
        #3) Lambda
        if wB is None:
            # All free assets
            return float((c4[i] c1*bi)/c),bi
        else:
            onesB=np.ones(wB.shape)
            l1=np.dot(onesB.T,wB)
            l2=np.dot(covarF_inv,covarFB)
            l3=np.dot(l2,wB)
            l2=np.dot(onesF.T,l3)
            return float(((1 l1+l2)*c4[i] c1*(bi+l3[i]))/c),bi
#
    def getMatrices(self,f):
        # Slice covarF,covarFB,covarB,meanF,meanB,wF,wB
        covarF=self.reduceMatrix(self.covar,f,f)
        meanF=self.reduceMatrix(self.mean,f,[0])
        b=self.getB(f)
        covarFB=self.reduceMatrix(self.covar,f,b)
        wB=self.reduceMatrix(self.w[1].reshape(1,1),b,[0])
        return covarF,covarFB,meanF,wB
```

```
#
    def getB(self,f):
        return self.diffLists(range(self.mean.shape[0]),f)
#
    def diffLists(self,list1,list2):
        return list(set(list1) set(list2))
#
    def reduceMatrix(self,matrix,listX,listY):
        # Reduce a matrix to the provided list of rows and columns
        if len(listX)==0 or len(listY)==0:return
        matrix_=matrix[:,listY[0]:listY[0]+1]
        for i in listY[1:]:
            a=matrix[:,i:i+1]
            matrix_=np.append(matrix_,a,1)
        matrix__=matrix_[listX[0]:listX[0]+1,:]
        for i in listX[1:]:
            a=matrix_[i:i+1,:]
            matrix__=np.append(matrix__,a,0)
        return matrix__
#
    def purgeNumErr(self,tol):
        # Purge violations of inequality constraints
#(associated with ill conditioned covar matrix)
        i=0
        while True:
            if i==len(self.w):break
            w=self.w[i]
            for j in range(w.shape[0]):
                if w[j] self.lB[j]< tol or w[j] self.uB[j]>tol:
                    del self.w[i]
                    del self.l[i]
                    del self.g[i]
                    del self.f[i]
                    break
            i+=1
```

```
#
    def purgeExcess(self):
        # Remove violations of the convex hull
        i,repeat=0,False
        while True:
            if repeat==False:i+=1
            if i==len(self.w)-1:break
            w=self.w[i]
            mu=np.dot(w.T,self.mean)[0,0]
            j,repeat=i+1,False
            while True:
                if j==len(self.w):break
                w=self.w[j]
                mu_=np.dot(w.T,self.mean)[0,0]
                if mu<mu_:
                    del self.w[i]
                    del self.l[i]
                    del self.g[i]
                    del self.f[i]
                    repeat=True
                    break
                else:
                    j+=1
#
    def getMinVar(self):
        # Get the minimum variance solution
        var=[]
        for w in self.w:
            a=np.dot(np.dot(w.T,self.covar),w)[0,0]
            var.append(a)
        return min(var)**.5,self.w[var.index(min(var))]
#
    def getMaxSR(self):
        # Get the max Sharpe ratio portfolio
        #1) Compute the local max SR portfolio between any two neighbor turning points
```

```
        w_sr, sr = [], []
        for i in range(len(self.w) 1):
            w0=np.copy(self.w[i])
            w1=np.copy(self.w[i+1])
            kargs={'minimum':False, 'args':(w0,w1)}
            a,b=self.goldenSection(self.evalSR,0,1,**kargs)
            w_sr.append(a*w0+(1 a)*w1)
            sr.append(b)
        return max(sr), w_sr[sr.index(max(sr))]
#
    def evalSR(self,a,w0,w1):
        # Evaluate SR of the portfolio within the convex combination
        w=a*w0+(1 a)*w1
        b=np.dot(w.T, self.mean)[0,0]
        c=np.dot(np.dot(w.T, self.covar),w)[0,0]**.5
        return b/c
#
    def goldenSection(self,obj,a,b,**kargs):
        # Golden section method. Maximum if kargs['minimum']==False is passed
        from math import log, ceil
        tol, sign, args=1.0e 9, 1, None
        if 'minimum' in kargs and kargs['minimum']==False: sign = 1
        if 'args' in kargs: args=kargs['args']
        numIter=int(ceil( 2.078087*log(tol/abs(b a))))
        r=0.618033989
        c=1.0 r
        # Initialize
        x1=r*a+c*b; x2=c*a+r*b
        f1=sign*obj(x1,*args); f2=sign*obj(x2,*args)
        # Loop
        for i in range(numIter):
            if f1>f2:
                a=x1
                x1=x2; f1=f2
                x2=c*a+r*b; f2=sign*obj(x2,*args)
```

```
            else:
                b=x2
                x2=x1; f2=f1
                x1=r*a+c*b; f1=sign*obj(x1,*args)
        if f1<f2:return x1, sign*f1
        else:return x2, sign*f2

    def efFrontier(self, points):
        # Get the efficient frontier
        mu, sigma, weights=[],[],[]
                                # remove the 1, to avoid duplications
        a=np.linspace(0,1,round(points/len(self.w)))[:1]
        b=range(len(self.w) 1)
        print("b=",b)
        print("a=",a)
        print("len=",len(self.w))
        for i in b:
            w0,w1=self.w[i], self.w[i+1]
# include the 1 in the last iteration
            if i==b[1]:a=np.linspace(0,1,round(points/len(self.w)))
            for j in a:
                print("j=",j)
                w=w1*j+(1 j)*w0
                print(w)
                weights.append(np.copy(w))
                mu.append(np.dot(w.T, self.mean)[0,0])
                sigma.append(np.dot(np.dot(w.T, self.covar),w)[0,0]**.5)
        return mu, sigma, weights

#Defining necessary functions. Details provided as comments

def generateData(nObs, size0, size1, sigma1):
    # Time series of correlated variables
    #1) generating some uncorrelated data
    np.random.seed(seed=12345);random.seed(12345)
```

```
    x=np.random.normal(0,1,size=(nObs,size0)) # each row is a variable
    #2) creating correlation between the variables
    cols=[random.randint(0,size0-1) for i in range(size1)]
    y=x[:,cols]+np.random.normal(0,sigma1,size=(nObs,len(cols)))
    x=np.append(x,y,axis=1)
    x=pd.DataFrame(x,columns=range(1,x.shape[1]+1))
    return x,cols

def fitKDE(obs,kernel='gaussian',x=None):
  # Fit kernel to a series of obs, and derive the prob of obs
  # x is the array of values on which the fit KDE will be evaluated
  if len(obs.shape)==1:obs=obs.reshape(-1,1)
  grid = GridSearchCV(KernelDensity(kernel = 'gaussian'), \
        {'bandwidth': np.linspace(0.1, 0.5, 20)}, cv = 5, iid = True)
  grid.fit(obs)
  bWidth=grid.best_params_['bandwidth']
  kde=KernelDensity(kernel=kernel,bandwidth=bWidth).fit(obs)
  if x is None:x=np.unique(obs).reshape(-1,1)
  if len(x.shape)==1:x=x.reshape(-1,1)
  logProb=kde.score_samples(x) # log(density)
  pdf=pd.Series(np.exp(logProb),index=x.flatten())
  return pdf

def mpPDF(var,q,pts):
  # q=T/N
  #calculating min max eigen value
  eMin,eMax=var*(1-(1./q)**.5)**2,var*(1+(1./q)**.5)**2
  #Return evenly spaced numbers over a specified interval
  eVal=np.linspace(eMin,eMax,pts)
  #pdf form
  pdf=q/(2*np.pi*var*eVal)*((eMax-eVal)*(eVal-eMin))**.5
  #into series
  pdf=pd.Series(pdf,index=eVal)
  return pdf
```

```
def errPDFs(var, eVal, q, pts=1000):
    pdf0 = mpPDF(var[0], q, pts) #theoretical pdf
    pdf1 = fitKDE(eVal, x=pdf0.index.values) #empirical pdf
    sse = np.sum((pdf1 pdf0)**2)
    return sse

def getRndCov(nCols,nFacts):
  w=np.random.normal(size=(nCols,nFacts))
  cov=np.dot(w,w.T) # random cov matrix, however not full rank
  cov+=np.diag(np.random.uniform(size=nCols)) # full rank cov
  return cov

def simCovMu(mu0,cov0,nObs,shrink=False):
  x=np.random.multivariate_normal(mu0.flatten(),cov0,size=nObs)
  mu1=x.mean(axis=0).reshape(1,1)
  if shrink:cov1=LedoitWolf().fit(x).covariance_
  else:cov1=np.cov(x,rowvar=0)
  return mu1,cov1

def findMaxEval(eVal,q):
    # Find max random eVal by fitting Marcenko s dist
    out=minimize(lambda *x:errPDFs(*x),.5,args=(eVal,q),bounds=((1E 5,1 1E 5),))
    if out['success']:var=out['x'][0]
    else:var=1
    eMax=var*(1+(1./q)**.5)**2
    return eMax,var

def getPCA(matrix):
  #Get eigen val and eigen vec from a Hermitian matrix
  eVal,eVec=np.linalg.eigh(matrix)
  #descending order of eval indices
  indices=eVal.argsort()[::1] #arguments for sorting eVal desc
  #reassigning eval and evac in descending order of eigen val
  eVal,eVec=eVal[indices],eVec[:,indices]
  #obtaining diagonal matrix form
```

```
    eVal=np.diagflat(eVal)
    return eVal,eVec

def denoisedCorr(eVal,eVec,nFacts):
    # Remove noise from corr by fixing random eigenvalues
    eVal_=np.diag(eVal).copy()
    eVal_[nFacts:]=eVal_[nFacts:].sum()/float(eVal_.shape[0] nFacts)
    eVal_=np.diag(eVal_)
    corr1=np.dot(eVec,eVal_).dot(eVec.T)
    corr1=cov2corr(corr1)
    return corr1

def denoisedCorr2(eVal,eVec,nFacts,alpha):
    # Remove noise from corr through targeted shrinkage
    eValL,eVecL=eVal[:nFacts,:nFacts],eVec[:,:nFacts]
    eValR,eVecR=eVal[nFacts:,nFacts:],eVec[:,nFacts:]
    corr0=np.dot(eVecL,eValL).dot(eVecL.T)

    corr1=np.dot(eVecR,eValR).dot(eVecR.T)
    corr2=corr0+alpha*corr1+(1 alpha)*np.diag(np.diag(corr1))
    return corr2

def deNoiseCov(cov0,q):
    corr0=cov2corr(cov0)
    eVal0,eVec0=getPCA(corr0)
    eMax0,var0=findMaxEval(np.diag(eVal0),q)
    nFacts0=eVal0.shape[0] np.diag(eVal0)[::1].searchsorted(eMax0)
    corr1=denoisedCorr(eVal0,eVec0,nFacts0)
    cov1=corr2cov(corr1,np.diag(cov0)**.5)
    corr2=denoisedCorr2(eVal0, eVec0, nFacts0,alpha=0.9)
    cov2=corr2cov(corr2, np.diag(cov0)**.5)
    corr3=denoisedCorr2(eVal0, eVec0, nFacts0,alpha=0.5)
    cov3=corr2cov(corr3, np.diag(cov0)**.5)
    corr4=denoisedCorr2(eVal0, eVec0, nFacts0,alpha=0.1)
    cov4=corr2cov(corr4, np.diag(cov0)**.5)
```

```
    return cov1,cov2,cov3,cov4

def correlDist(corr):
    # A distance matrix based on correlation, where 0<=d[i,j]<=1
    # This is a proper distance metric
    dist=((1 corr)/2.)**.5 # distance matrix
    return dist

def getIVP(cov,**kargs):
    # Compute the inverse variance portfolio
    ivp=1./np.diag(cov)
    ivp/=ivp.sum()
    return ivp

def getQuasiDiag(link):
    # Sort clustered items by distance
    link=link.astype(int)
    sortIx=pd.Series([link[1,0],link[1,1]])
    numItems=link[1,3] # number of original items
    while sortIx.max()>=numItems:
        sortIx.index=range(0,sortIx.shape[0]*2,2) # make space
        df0=sortIx[sortIx>=numItems] # find clusters
        i=df0.index;j=df0.values numItems
        sortIx[i]=link[j,0] # item 1
        #REPRODUCING THE NUMERICAL EXAMPLE 241
        df0=pd.Series(link[j,1],index=i+1)
        sortIx=sortIx.append(df0) # item 2
        sortIx=sortIx.sort_index() # re sort
        sortIx.index=range(sortIx.shape[0]) # re index
    return sortIx.tolist()

def getClusterVar(cov,cItems):
    # Compute variance per cluster
    cov_=cov.loc[cItems,cItems] # matrix slice
    w_=getIVP(cov_).reshape(1,1)
```

```
    cVar=np.dot(np.dot(w_.T,cov_),w_)[0,0]
    return cVar

def getRecBipart(cov,sortIx):
    # Compute HRP alloc
    w=pd.Series(1,index=sortIx)
    cItems=[sortIx] # initialize all items in one cluster
    while len(cItems)>0:
        cItems=[i[j:k] for i in cItems for j,k in ((0,int(len(i)/2)), \
        (int(len(i)/2),int(len(i)))) if len(i)>1] # bi section
        for i in range(0,len(cItems),2): # parse in pairs
            cItems0=cItems[i] # cluster 1
            cItems1=cItems[i+1] # cluster 2
            cVar0=getClusterVar(cov,cItems0)
            cVar1=getClusterVar(cov,cItems1)
            alpha=1 cVar0/(cVar0+cVar1)
            w[cItems0]*=alpha # weight 1
            w[cItems1]*=1 alpha # weight 2
    return w

def plotCorrMatrix(path,corr,labels=None):
    # Heatmap of the correlation matrix
    if labels is None:labels=[]
    plt.pcolor(corr)
    plt.colorbar()
    plt.yticks(np.arange(.5,corr.shape[0]+.5),labels)
    plt.xticks(np.arange(corr.shape[0]+.5,.5),labels)
    plt.savefig(path)
    plt.clf();plt.close() # reset pylab
    return

def clusterKMeansBase(corr0,maxNumClusters=None,n_init=10):
    dist,silh=((1 corr0.fillna(0))/2.)**.5,pd.Series() # distance matrix
    if maxNumClusters is None:maxNumClusters=int(corr0.shape[0]/2)
    for init in range(n_init):
```

```
        print("init=",init)
        for i in range(2,maxNumClusters+1):
            kmeans_=KMeans(n_clusters=i,n_init=10)
            kmeans_=kmeans_.fit(dist)
            silh_=silhouette_samples(dist,kmeans_.labels_)
            stat=(silh_.mean()/silh_.std(),silh.mean()/silh.std())
            if np.isnan(stat[1]) or stat[0]>stat[1]:
                silh,kmeans=silh_,kmeans_
    newIdx=np.argsort(kmeans.labels_)
    corr1=corr0.iloc[newIdx] # reorder rows
    corr1=corr1.iloc[:,newIdx] # reorder columns
    clstrs={i:corr0.columns[np.where(kmeans.labels_==i)[0]].tolist() \
                    for i in np.unique(kmeans.labels_)} # cluster members
    silh=pd.Series(silh,index=dist.index)
    return corr1,clstrs,silh

def optPort_nco(cov,mu=None,maxNumClusters=None):
    cov=pd.DataFrame(cov)
    if mu is not None:mu=pd.Series(mu[:,0])
    corr1=cov2corr(cov)
    print("K begin")
    corr1,clstrs,_=clusterKMeansBase(corr1,maxNumClusters,n_init=10)
    print(clstrs)
    print("K done")
    wIntra=pd.DataFrame(0
            =cov.index,columns=clstrs.keys())
    for i in clstrs:
        cov_=cov.loc[clstrs[i],clstrs[i]].values
        mu_=(None if mu is None else mu.loc[clstrs[i]].values.reshape(1,1))
        wIntra.loc[clstrs[i],i]=optPortcon(cov_,mu_).flatten()
    cov_=wIntra.T.dot(np.dot(cov,wIntra)) # reduce covariance matrix
    mu_=(None if mu is None else wIntra.T.dot(mu))
    wInter=pd.Series(optPortcon(cov_,mu_).flatten(), \
            index=cov_.index)
    nco=wIntra.mul(wInter,axis=1).sum(axis=1).values.reshape(1,1)
```

```
    return nco


def optPort_hrp(cov,mu=None):
    #HRP results
    corrh=cov2corr(cov)
    #3) cluster
    dist=correlDist(corrh) #obtaining distance matrix
    link=sch.linkage(dist,'single')
    sortIx=getQuasiDiag(link)
    sortIx=corrh.index[sortIx].tolist() # recover labels
    #df0=corrh.loc[sortIx,sortIx] # reorder
    #plotCorrMatrix('HRP3_corr1.png',df0,labels=df0.columns)
    hrp=getRecBipart(pd.DataFrame(cov),sortIx)
    return hrp


def monteCarlo(mu0,cov0,nObs,nSims,minVarPortf,shrink):
    w1=pd.DataFrame(0,columns=range(cov0.shape[0]),index=range(nSims),dtype=float)
    w1_nco=w1.copy(deep=True)
    w1_hrp=w1.copy(deep=True)
    w1_ivp=w1.copy(deep=True)
    w1_cla=w1.copy(deep=True)
    for i in range(nSims):
        print("nSims=",i)
        mu1,cov1=simCovMu(mu0,pd.DataFrame(cov0),nObs,shrink)
        cov1,_,_,_=deNoiseCov(cov1,nObs*1./cov1.shape[1])
        cla=CLA_3(mu1,cov1,lb,ub)
        cla.solve()
        w1_cla.loc[i]=cla.getMinVar()[1].flatten()
        if minVarPortf:mu1=None
        w1.loc[i]=optPortcon(cov1,mu1).flatten()
        w1_hrp.loc[i]=optPort_hrp(pd.DataFrame(cov1),mu1)
        w1_ivp.loc[i]=getIVP(cov1)
        w1_nco.loc[i]=optPort_nco(cov1,mu1,int(cov1.shape[0]/2)).flatten()
    return w1,w1_nco,w1_hrp,w1_ivp,w1_cla
```

```
def RMSE(w1,w0):
    mean_rmse=np.mean((w1 w0).values.flatten()**2)**.5
    return mean_rmse


def DataTestOpt(returns):
    train_index=returns.shape[0] 180
    data_shape=returns.shape[0]
    returns_test=returns.iloc[train_index:]
    w3=pd.DataFrame(0,columns=returns_test.columns,index=range(228),dtype=float)
    w3_nco=w3.copy(deep=True)
    w3_hrp=w3.copy(deep=True)
    w3_ivp=w3.copy(deep=True)
    w3_cla=w3.copy(deep=True)
    port_return=pd.DataFrame(0,columns=["MVO","NCO","HRP","IVP","CLA"]\
                    ,index=w3.index,dtype=float)
    port_sd=port_return.copy(deep=True)
    port_sharpe=port_return.copy(deep=True)
    for i in range(data_shape train_index):
        ret_input=returns[0+i:train_index+i]
        print(returns[0+i:train_index+i])
        cov3=np.array(returns[0+i:train_index+i].cov())
        mu3=np.array(returns[0+i:train_index+i].mean())
        mu3=mu3.reshape(mu3.shape[0],1)
        cov3,_,_,_=deNoiseCov(cov3,train_index*1./cov3.shape[1])


        cov3eval,cov3evec=getPCA(cov2corr(cov3))
        newvec=np.dot(ret_input,cov3evec)
        newvec[:,0:1]=np.zeros((ret_input.shape[0],1))
        rvec=np.dot(newvec,cov3evec)
        cov3_detoned=pd.DataFrame(rvec).cov()


        lb = np.zeros(mu3.shape)
        ub = np.ones(mu3.shape)
        cla=CLA_3(mu3,cov3,lb,ub)
        cla.solve()
```

```
        w3_cla.loc[i]=cla.getMinVar()[1].flatten()
        w3.loc[i]=optPortcon(cov3,None).flatten()
        w3_hrp.loc[i]=optPort_hrp(pd.DataFrame(cov3),None).values
        w3_ivp.loc[i]=getIVP(cov3)
        w3_nco.loc[i]=optPort_nco(cov3_detoned,None,int(cov3.shape[0]/2)).flatten()
        port_return.loc[i]=[np.array(w3.loc[i]).dot(mu3), \
                            np.array(w3_nco.loc[i]).dot(mu3), \
                                np.array(w3_hrp.loc[i]).dot(mu3), \
                                    np.array(w3_ivp.loc[i]).dot(mu3), \
                                        np.array(w3_cla.loc[i]).dot(mu3),]

    return w3,w3_hrp,w3_ivp,w3_nco,w3_cla,port_return

##################################################
# Test Preliminaries: Obtaining initial inputs #
##################################################

#We compare the techniques used for treating the covariance matrix
#namely, Ledoit Wolfe Shrinkage, Eigen Value Clipping and Targeted Shrinkage

#Generating initial data X0~(mu0,cov0) for testing
#nBlocks  number of blocks
#bSize  number of assets in each block
#bCorr  correlation within blocks

nBlocks,bSize,bCorr=10,10,.5
np.random.seed(100)

#Obtaining generated data mean and covariance matrix
mu0,cov0=formTrueMatrix(nBlocks,bSize,bCorr)

#Initializing testing parameters
#nObs number of observations to generate for simulated data
#nTrials  number of simulations to perform
#shrinkT  decides where Ledoit Wolf shrinkage will be performed
```

```
nObs,nTrials,shrinkT=10000,1000,True

corr0=cov2corr(cov0)
eVal0,eVec0=getPCA(corr0)
eMax0,var0=findMaxEval(np.diag(eVal0),nObs/(nBlocks*bSize))
nFacts0=eVal0.shape[0] np.diag(eVal0)[::1].searchsorted(eMax0)



##########################################
# Test 1: Comparing Denoising Techniques #
##########################################

#Obtaining actual weight of generated data X0 via standard MVO
#w0_min= Minimim Variance Portfolio weight
#w0_max= Maximum Sharpe Ratio Portfolio weight
w0_min=optPortcon(cov0,None)
w0_max=optPortcon(cov0,mu0)

#Initializing simulated output dataframe
#w1=Neither denoised nor shrunk
#w1_EC=Denoised via Eigen Clipping but no Shrinkage
#w1_TS=Denoised via Targeted shrinkage but no Shrinkage
#w1s= Not denoised but shrunk
#w1s_EC=Denoised via Eigen Clipping and shrunk
#w1s_TS=Denoised via Targeted shrinkage and shrunk
w1=pd.DataFrame(columns=range(cov0.shape[0]),index=range(nTrials),dtype=float)
w1_EC=w1.copy(deep=True)
w1_TS=w1.copy(deep=True)
w1_TS2=w1.copy(deep=True)
w1_TS3=w1.copy(deep=True)



w1s=w1.copy(deep=True)
w1s_EC=w1.copy(deep=True)
w1s_TS=w1.copy(deep=True)
```

```
w1s_TS2=w1.copy(deep=True)
w1s_TS3=w1.copy(deep=True)


#Creating dataframe with actual weight estimates for comparison
w0_min_test=np.repeat(w0_min.T,w1.shape[0],axis=0)
w0_max_test=np.repeat(w0_max.T,w1.shape[0],axis=0)

#Obtaining simulated weight estimates

#Minimum Variance Portfolio Weights
np.random.seed(100)
#Simulated estimates with and without Shrinking and Denoising
w1,w1_EC,w1_TS,w1_TS2,w1_TS3,w1s,w1s_EC,w1s_TS,w1s_TS2,w1s_TS3= \
    DenoiseTest(mu0, cov0, nObs, nTrials, minVarPortf=True)

#Calculating RMSE
result=[[RMSE(w1,w0_min_test),RMSE(w1_EC,w0_min_test),RMSE(w1_TS,w0_min_test), \
        RMSE(w1_TS2,w0_min_test),RMSE(w1_TS3,w0_min_test)], \
        [RMSE(w1s,w0_min_test),RMSE(w1s_EC,w0_min_test),RMSE(w1s_TS,w0_min_test), \
         RMSE(w1s_TS2,w0_min_test),RMSE(w1s_TS3,w0_min_test)]]
MinVar_RMSE=pd.DataFrame(result,index=["No_shrinkage","Shrinkage"], \
                         columns=["Direct","EC","TS0.9","TS0.5","TS0.1"])
#Printing RMSE obtained for Minimum Variance Weights
print(MinVar_RMSE)

plt1=MinVar_RMSE.plot(kind='bar')
plt1.title('Denoising_techniques_for_Minimum_Variance_Portfolio')
plt1.ylabel('Standard_Deviation_of_RMSE')
current_handles, _ = plt.gca().get_legend_handles_labels()
plt1.legend(loc='lower_right')

#Maximum Sharpe Ratio Portfolio Weights
np.random.seed(100)
#Simulated estimates with and without Shrinking and Denoising
```

```
w1,w1_EC,w1_TS,w1_TS2,w1_TS3,w1s,w1s_EC,w1s_TS,w1s_TS2,w1s_TS3= \
    DenoiseTest(mu0, cov0, nObs, nTrials, minVarPortf=False)

#Calculating RMSE
result=[[RMSE(w1,w0_max_test),RMSE(w1_EC,w0_max_test),RMSE(w1_TS,w0_max_test), \
        RMSE(w1_TS2,w0_max_test),RMSE(w1_TS3,w0_max_test)], \
        [RMSE(w1s,w0_max_test),RMSE(w1s_EC,w0_max_test),RMSE(w1s_TS,w0_max_test), \
        RMSE(w1s_TS2,w0_max_test),RMSE(w1s_TS3,w0_max_test)]]
MaxSharpe_RMSE=pd.DataFrame(result,index=["No_shrinkage","Shrinkage"], \
                            columns=["Direct","EC","TS0.9","TS0.5","TS0.1"])
#Printing RMSE obtained for Maximum Sharpe Ratio weights
print(MaxSharpe_RMSE)

plt2=MaxSharpe_RMSE.plot(kind='bar')
plt.title('Denoising_techniques_for_Maximum_Sharpe_Ratio_Portfolio')
plt.ylabel('Standard_Deviation_of_RMSE')
current_handles, _ = plt.gca().get_legend_handles_labels()
plt1.legend(loc='lower_right')

#####################################################################
# Test 2: Comparing Optimization Techniques on Artifical Data #
#####################################################################

#Denoising covariance matrix
#cov0_den,_,_,_=deNoiseCov(cov0,nObs*1./cov0.shape[1])
#Obtaining actual weights from generated data X0
#Standard MVO weight
w20=optPortcon(cov0,None).T
#Nested Clustered Optimization weight
w20_nco=np.array(optPort_nco(cov0,None,int(cov0.shape[0]/2)))
#Hierarchical Risk Parity weight
w20_hrp=np.array(optPort_hrp(pd.DataFrame(cov0),mu0))
#Inverse Parity Portfolio weight
w20_ivp=np.array([getIVP(cov0)])
#Critical Line Algorithm weight
```

```
lb = np.zeros(mu0.shape)
ub = np.ones(mu0.shape)
cla=CLA_3(mu0,np.array(cov0),lb,ub)
cla.solve()
w20_cla=cla.getMinVar()[1]

#Assigning number of simulations to run
nSims=1000

#Obtaining simulated weights of over nSims simulations
#We apply Eigen clipping denoising procedure without shrinkage
w2,w2_nco,w2_hrp,w2_ivp,w2_cla=monteCarlo(mu0,cov0,nObs=10000, \
                        nSims=nSims,minVarPortf=True,shrink=False)

#Creating dataframe with actual weight estimates for comparison
w20_cla_test=np.repeat(w20_cla.T,nSims,axis=0)

#Obtaining mean RMSE values of optimization methods over generated data X0
result2=[[RMSE(w2,w20_cla_test),RMSE(w2_nco,w20_cla_test), \
          RMSE(w2_hrp,w20_cla_test),RMSE(w2_ivp,w20_cla_test), \
                                    RMSE(w2_cla,w20_cla_test)]]
#Converting results to dataframe
result2=pd.DataFrame(result2,index=["RMSE"],columns=["MVO","NCO","HRP","IVP","CLA"])
#Printing results
print(result2)

np.dot(w2 w20_cla_test,mu0).std()
np.dot(w2_cla w20_cla_test,mu0).std()
np.dot(w2_hrp w20_cla_test,mu0).std()
np.dot(w2_nco w20_cla_test,mu0).std()

w2_std_val=pd.DataFrame(columns=["MVO","NCO","HRP","CLA"],index=range(1000))
for i in range(0,1000):
    w2_std_val.iloc[i,0]=np.array((w2 w20_cla_test).loc[i]).reshape(1,100) \
                        .dot(cov0).dot(np.array((w2 w20_cla_test).loc[i])) \
```

```
                                                    reshape(100,1))
    w2_std_val.iloc[i,1]=np.array((w2_nco  w20_cla_test).loc[i]).reshape(1,100) \
                        .dot(cov0).dot(np.array((w2  w20_cla_test).loc[i]) \
                                                    .reshape(100,1))
    w2_std_val.iloc[i,2]=np.array((w2_hrp  w20_cla_test).loc[i]).reshape(1,100) \
                        .dot(cov0).dot(np.array((w2  w20_cla_test).loc[i]) \
                                                    .reshape(100,1))
    w2_std_val.iloc[i,3]=np.array((w2_cla  w20_cla_test).loc[i]).reshape(1,100)\
                        .dot(cov0).dot(np.array((w2  w20_cla_test).loc[i]) \
                    .reshape(100,1))

var_w2=w2_std_val.std(0)
np.dot(w2  w20_cla_test,mu0)/((w2_std_val**.5).loc[0])
np.dot(w2_cla  w20_cla_test,mu0).std()/(var_w2.iloc[3]**.5)
np.dot(w2_hrp  w20_cla_test,mu0).std()/(var_w2.iloc[2]**.5)
np.dot(w2_nco  w20_cla_test,mu0).std()/(var_w2.iloc[1]**.5)

#####################################################################
# Test 3: Comparing Optimization Techniques Performance on S&P100 #
#####################################################################

# Ticker symbols of stocks in S&P100
tickers=['AAPL', 'ABT', 'ADBE', 'AIG', 'ALL', 'AMGN', 'AMT', 'AMZN', 'AXP', 'BA', \
         'BAC', 'BIIB', 'BK', 'BKNG', 'BLK', 'BMY', 'BRK B', 'C', 'CAT', 'CL', \
         'CMCSA', 'COF', 'COP', 'COST', 'CSCO', 'CVS', 'CVX', 'DD', 'DHR', 'DIS', \
         'DUK', 'EMR', 'EXC', 'F', 'FDX', 'GE', 'GILD', 'GS', 'HD', 'HON', 'IBM', \
         'INTC', 'JNJ', 'JPM', 'KO', 'LLY', 'LMT', 'LOW', 'MCD', 'MDT']
start = datetime.date(2000,1,1)
end = datetime.date(2020,1,1)
index = pd.date_range(start,end,freq='D')
data = pd.DataFrame(index=index,columns=tickers)
data = data.fillna(0)

for tick in tickers:
    print(tick)
```

```
    data[tick]= web.DataReader(tick,'yahoo', start, end)['Adj_Close']
    print(tick)
print(data)

#removing observations on inactive days (NaN rows)
data=data.dropna(0,how='all')
data=data.dropna(1,how='any')
data

returns= data.resample('M').ffill().pct_change()
returns=returns.dropna(0,how='all')

w3,w3_hrp,w3_ivp,w3_nco,w3_cla,port_return,port_sd=DataTestOpt(returns)
```